

Blockchain Notebook – Bitcoin

Xuan Fu

xuanf@usc.edu

Table of Contents

INTRODUCTION	3
BITCOIN CRYPTOGRAPHY	4
HASHING	4
DIGITAL SIGNATURE	7
BITCOIN DATA STRUCTURE	10
HASH POINTER	10
MERKLE TREE	12
BITCOIN PROTOCOL	14
DIGITAL CURRENCY SYSTEM DESIGN	14
BITCOIN CONSENSUS	19
BITCOIN SYSTEM IMPLEMENTATION	24
UTXO	24
BLOCK EXAMPLE	25
BITCOIN MINING	31
MEMORYLESS	31
TOTAL SUPPLY	32
SYSTEM SAFETY	33
MINING DIFFICULTY	38
MINER & MINING POOL	42
BITCOIN NETWORK	49
P2P NETWORK	50
FULL NODE & LIGHT NODE	55
BITCOIN SCRIPT	58
SCRIPT LANGUAGE	58
SCRIPT TYPES	65
MULTI-SIGNATURE	70
PROOF OF BURN	74
BITCOIN FORKING	77
HARD FORK	77
SOFT FORK	80
BITCOIN ANONYMITY	84
ANONYMITY SABOTAGE	84
ANONYMITY REINFORCEMENT	86
ZERO-KNOWLEDGE PROOF & BLIND SIGNATURE	87
REFERENCE	90

Introduction

I decided to create this notebook because I encountered many detours on my journey to understanding blockchain. Blockchain-related knowledge is scattered across the Internet, making it difficult for the general public to develop a systematic understanding of the subject. I hope that by reading and studying this notebook, readers will be able to quickly understand blockchain and its tremendous potential.

My personal understanding of blockchain is as follows:

A blockchain is a linked-list data structure connected by hash “pointers.” It provides strong traceability and resistance to tampering. Bitcoin’s success is due not only to innovations in computer technology, but also to the demand for reform in social ideology and the financial system. The emergence of blockchain has challenged the current top-down, centralized organizational structure and made decentralized consensus structures possible.

Blockchain is an interdisciplinary field that spans computer science, cryptography, economics, finance, and politics. Although blockchain has great potential, it is not a “panacea.” We should not try to solve every problem with blockchain.

In this notebook, I will try to explain blockchain technology in plain language. When professional or technical terms are introduced, visual explanations—presented in bold and italicized text—will be provided to help readers who do not have a background in computer science.

Bitcoin Cryptography

Cryptocurrencies such as Bitcoin are not encrypted in the conventional sense. Instead, Bitcoin uses an unencrypted ledger that is publicly accessible and contains information such as addresses and transactions. The Bitcoin system primarily uses two concepts from cryptography: **hashing** and **digital signatures**.

Hashing

Here, “hashing” refers to the use of cryptographic hash functions. It should not be confused with the general-purpose hash functions used in hash tables.

A hash table is a data structure in computer science that improves the efficiency of data retrieval. You can think of it as a way of organizing items so that you can quickly find what you need later. For example, imagine that you have many items to store in a series of numbered drawers.

A general-purpose hash function can be viewed as a black-box processor. You put an item into it, and the black box outputs a drawer number. This number is called a hash code. You can then use the number to find the corresponding drawer.

A cryptographic hash function has three important properties. The first is called **collision resistance**. This does not mean that hash collisions can never occur. Rather, collision resistance means that it is computationally infeasible to deliberately and efficiently find such a collision. In other words, given $x \neq y$, the only practical approach is to use brute force to find a pair of values for which:

$$\text{Hash}(x) = \text{Hash}(y)$$

In practice, such a brute-force search is computationally infeasible for a secure cryptographic hash function.

A hash collision occurs when two different inputs are put into the black-box processor and it outputs the same hash value—or drawer number—for both inputs. If both items are placed in the same drawer, they “collide”; hence the term hash collision. A well-designed hash function should make it extremely difficult to find such a collision.

In computer science, brute force refers to an exhaustive search: trying every possible option one by one until the correct answer is found.

What is the practical significance of collision resistance? Imagine that we have a message and put it into a hash function to obtain its hash value:

$$\text{Hash}(\text{message}) = 0000741$$

If someone modifies the message and turns it into message', we can put the modified message into the same hash function:

Hash(message') = 0000527

We can see that the hash value has changed. By comparing the hash values, we can detect whether the original message has been tampered with.

Let us consider another scenario. Suppose you upload a file containing your bank account information to a cloud storage server and download it again in the future. How can you ensure that the downloaded file is exactly the same as the one you originally uploaded and has not been tampered with? The answer is simple: compare the file's hash value before uploading it with its hash value after downloading it.

Note that collision resistance is not usually proven in an absolute mathematical sense. Instead, confidence in this property is based largely on extensive analysis and practical experience: cryptography experts have not found an efficient method for generating collisions in secure cryptographic hash functions. However, if you implement a very simple hash function yourself, it may be easy to break. For example, researchers have successfully generated collisions for the MD5 hash function. Many different hash functions have been developed. Some have withstood extensive analysis and the test of time, and no practical collision attacks against them are currently known.

The second property of a cryptographic hash function is called **hiding**. This means that it should be computationally infeasible to reverse the hashing process. You can put an object into a hash function and obtain its hash value, but you cannot practically reconstruct the original object from that value alone. In theory, we could attempt to find the original object through brute force, but this is impractical in most cases when the input is sufficiently unpredictable.

The hiding property can be compared to driving from a starting point to a destination along a one-way road. After arriving at the destination, you cannot simply travel backward along the same road to discover where the journey began.

By combining **collision resistance** with the **hiding** property, we can create a **cryptographic commitment**, which is the digital equivalent of a sealed envelope.

For example, suppose a stock market expert wants to predict whether a particular stock will rise or fall the next day. If the expert announces the prediction publicly on television, the announcement could affect the market, making it difficult to evaluate the prediction objectively. Instead, we could ask the expert to place the prediction in a sealed envelope in advance. After the stock market closes the next day, the envelope could be opened to determine whether the prediction was accurate. How can we achieve the same result digitally?

We can enter the prediction into a hash function to obtain a hash value. Because of the hiding property, we can safely publish the hash value without revealing the prediction. The next day, we announce the original prediction. Because of collision resistance, others can verify that the announced prediction is the same one that was committed to the previous day.

However, the hiding property requires the input space to be sufficiently large and unpredictable. For example, suppose we predict that a particular stock will increase by 10% the next day. We cannot simply enter the stock's name into the hash function and publish the resulting hash value. Other people could easily determine which stock name produced the hash value by using brute force, because there are only a few thousand listed stocks. They could hash the name of every listed stock and compare the results with the published hash value.

Therefore, we usually add a sufficiently large random value to the original input to make a brute-force attack much more difficult:

Hash(object + random value)

The third property is called **puzzle-friendliness**. Informally, this means that the output of a secure cryptographic hash function is unpredictable. Without trying different inputs, you cannot predict which input will produce a hash value within a specified target range. To obtain such a hash value, you generally have to try many different inputs one by one. This process can require a great deal of computational work.

What is this third property used for? It plays an important role in Bitcoin mining. A block header contains several fields, one of which is the **nonce**. In simplified terms, mining involves repeatedly changing the nonce and hashing the block header until a hash value that falls within the target range is found, as illustrated in Figure 1.

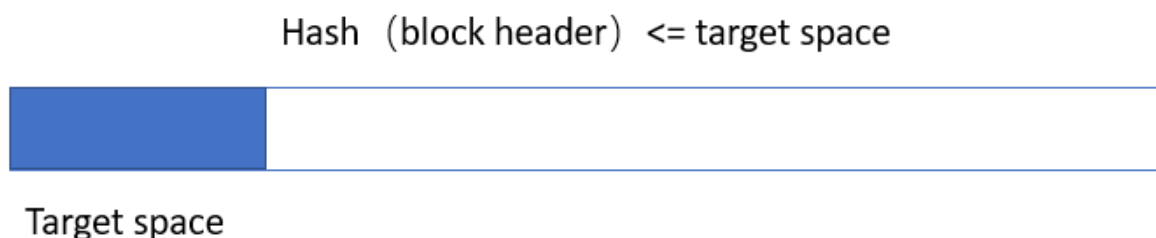


Fig 1

The puzzle-friendly property ensures that this trial-and-error process requires a substantial amount of computational work. For this reason, it is called **proof of work**.

During the mining process, when a node finds a nonce that satisfies the requirements, it packages the nonce into a block and broadcasts the block to the Bitcoin network. After receiving the block, other nodes extract the nonce and use the hash function to verify whether it produces a valid result. The verification process is relatively simple. Therefore, mining is designed so that finding a solution is difficult, while verifying one is easy.

The hash function used by Bitcoin for proof of work is SHA-256, which stands for **Secure Hash Algorithm 256-bit**. SHA-256 provides the three properties discussed above: collision resistance, hiding, and puzzle-friendliness.

Digital Signature

We have now finished discussing hashing, so it is time to move on to digital signatures. Before doing so, however, let us first discuss Bitcoin accounts.

In everyday life, if we want to open a bank account, we usually have to visit a bank and present identification documents. This is a centralized process. In the Bitcoin system, however, anyone can locally generate a pair of cryptographic keys: a **public key** and a **private key**. This key pair allows the user to control bitcoin and authorize transactions. The concepts of public and private keys come from asymmetric cryptography.

The concepts of public keys, private keys, and symmetric and asymmetric encryption all come from cryptography—a collection of techniques used to protect information and secure communications.

The main characteristics of public-key cryptography are as follows:

- a) A private key has a corresponding public key.
- b) Different private keys produce different public keys, except with a negligible probability of an accidental collision.
- c) Depending on the cryptographic system, the public and private keys can be used for encryption and decryption or for signing and verification.
- d) It is easy to generate a private key and derive its corresponding public key, but it is computationally infeasible to derive the private key from the public key.

Historically, **symmetric encryption** was developed before **asymmetric encryption**. In symmetric encryption, the sender and the receiver share the same secret key. The sender uses this key to encrypt a message, and the receiver uses the same key to decrypt it.

This system requires a secure method of distributing the secret key to the parties that want to communicate. Key distribution is inconvenient because we must ensure that the key is not intercepted or stolen while it is being delivered.

Asymmetric encryption helps address this problem by using a public key for encryption and a private key for decryption. If A wants to send a message to B, A encrypts the message using B's public key. After receiving the encrypted message, B decrypts it using B's private key.

Because a public key does not need to be kept secret, it can be shared with everyone. A person who wants to send you a confidential message can encrypt it using your public key. After receiving the message, you decrypt it locally using your private key. When you want to reply, you encrypt your response using the other person's public key. The recipient can then decrypt it using their own private key. Neither party needs to know the other party's private key, which helps solve the key-distribution problem found in symmetric encryption systems.

To understand this concept, imagine that you and a friend are exchanging private letters and do not want the postal carrier to read them. You come up with the following solution: each of you buys several padlocks, representing your respective public keys, and gives those padlocks to the other person. Each of you keeps the key that opens your own padlocks, representing your private key.

After writing a letter to your friend, you place it in a box and secure the box using one of the padlocks they gave you. This padlock represents their public key. You then give the locked box to the postal carrier. When your friend receives it, they use their own key—their private key—to unlock the box and read the letter.

When your friend writes a reply, they place it in a box and secure the box using one of the padlocks you gave them, representing your public key. After receiving the box, you use your own key—your private key—to unlock it and read the reply.

Throughout this process, neither of you needs to reveal your own key, or private key. Your padlocks, or public keys, can be freely distributed. It does not matter if other people obtain the padlocks, because public keys are intended to be publicly available.

In the Bitcoin system, we can begin participating locally by generating a public-key and private-key pair. The public key is somewhat similar to an account number, while the private key is somewhat similar to an account password. However, a private key does more than provide access: it is used to authorize transactions by generating digital signatures.

To prevent impersonation, transaction information and a digital signature are produced and distributed as follows:

1. Suppose I want to send 10 bitcoins to Bob. I first create the transaction information.
2. A hash of the transaction data is calculated to produce a digest. I then use my private key and a digital-signature algorithm to generate a signature for that transaction.
3. I broadcast the transaction information and its signature to the Bitcoin network.
4. After receiving the transaction and its signature, other nodes use the corresponding public key to verify the signature. A valid signature provides evidence that the

transaction was authorized by the holder of the private key. The verification process also confirms that the signed transaction data has not been modified.

xuanf@usc.edu

Bitcoin Data Structure

Hash Pointer

A conventional **pointer** stores the memory address of an object. A **hash pointer** contains not only a reference to an object but also the hash value of the data being referenced. We use $H()$ to represent a hash function. A hash pointer allows us to locate the referenced object and verify whether its data has been tampered with.

A data structure is a concept in computer science. Common data structures include arrays, linked lists, heaps, stacks, queues, hash tables, and trees. There is no need to be intimidated by these terms. Data structures are essentially digital abstractions of ways in which we organize things in everyday life, such as the hash table discussed in the previous chapter.

In essence, a data structure defines how a computer organizes and stores data. For example, a queue data structure resembles a line of people waiting for service. Similarly, when you place one plate on top of another after washing the dishes, the resulting arrangement resembles a stack. Each data structure is best suited to particular types of tasks.

A pointer is a special type of variable used in some programming languages. Its purpose is to store a memory address. Computer memory is divided into locations, each of which has a corresponding address.

You can imagine computer memory as a hotel in which each room has a room number. The room number represents the memory address. Suppose the hotel has ten rooms, numbered from 01 to 10. We define a variable A , which we can imagine as a person, and assign it the value 10: $A=10$

Suppose A is assigned to room 03. The person representing A enters room 03 and leaves a card bearing the number 10 inside it.

Next, we define a pointer variable A' , which we can imagine as another person. A' finds out which room contains A 's card and writes the room number, 03, on a piece of paper. A' then places that paper in an available room, such as room 09. The paper stored in room 09 contains the address of A . In this sense, we say that a pointer stores a memory address.

The fundamental data structure underlying Bitcoin is the blockchain, which can be understood as a structure similar to a **linked list**. An ordinary linked list connects its elements using conventional pointers or references, whereas a blockchain connects its blocks through cryptographic hashes. In this conceptual model, a blockchain can be viewed as a linked structure that uses hash pointers.

Hash pointers can be used in place of ordinary references when the resulting structure is acyclic—that is, when following its references cannot lead back to an earlier point in a loop. For example, a singly linked list can use hash pointers, whereas a circular linked list cannot be

constructed in the same straightforward way because its circular dependencies would make the hash values impossible to determine independently.

A linked list is a basic data structure in computer science. It consists of nodes, with each node connected to the next one. Every node can also store data.

You can imagine a line of people, with each person representing a node. Each person places a hand on the shoulder of the next person, creating a series of links. Every person also carries a backpack, representing the data stored in that node. Each backpack can contain different kinds of information.

The first block in a blockchain is called the **genesis block**, while the block most recently added to the blockchain is called the latest block or the most recent block. Each block contains the hash of the previous block, thereby linking it to its predecessor, as illustrated in Figure 2.

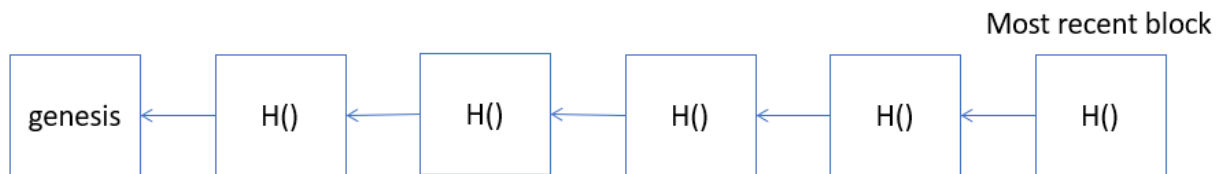


Fig 2

Each block contains a hash calculated from the previous block's header, which itself contains the hash of the block before it. As a result, every block indirectly commits to the entire preceding history of the blockchain. This structure makes it easy to detect whether a previous block has been tampered with, making the blockchain a **tamper-evident log**. If any part of a previous block is altered, its hash will change, breaking the links between that block and all subsequent blocks. The effects of the modification will therefore propagate through the chain to the latest block, as illustrated in Figure 3.

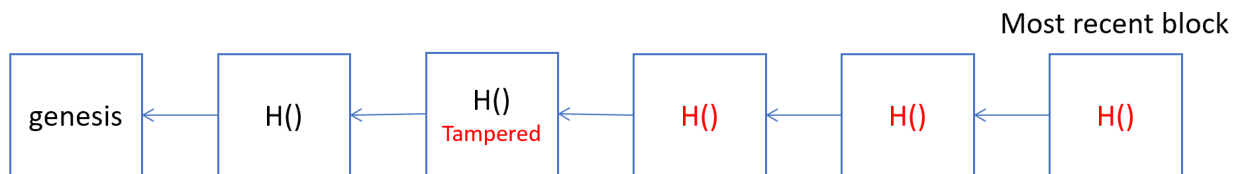


Fig 3

Therefore, we only need to observe the H() of the most recent block to check whether any block has been tampered with. Because of this feature, some nodes in the Bitcoin system do not need to store the contents of the entire chain; they only need to store the last few thousand blocks. When necessary, they can ask other nodes in the system for help. Even if a

malicious node provides you with a tampered blockchain, you can easily detect it by checking the hash values.

Merkle Tree

Another data structure used in Bitcoin is the Merkle tree (Figure 4). The difference between a Merkle tree and an ordinary binary tree is the use of hash pointers.

A tree is a basic data structure in computer science. You can think of it as your own family tree. It looks like a tree growing upside down, doesn't it? Starting with your ancestors, it continues to grow by forming branches. In a binary tree, each node can have no more than two children, and each of those children can also have no more than two children, and so on.

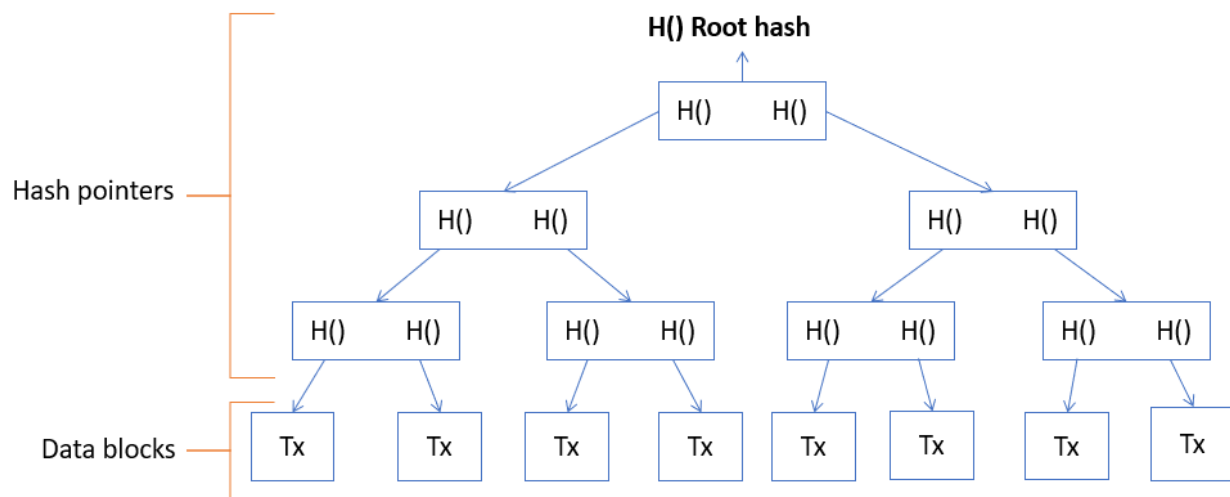


Fig 4

The benefits of the Merkle tree data structure are the same as those of a hash-pointer linked list. As long as we remember the correct root hash value, we can detect whether the information in the tree has been tampered with.

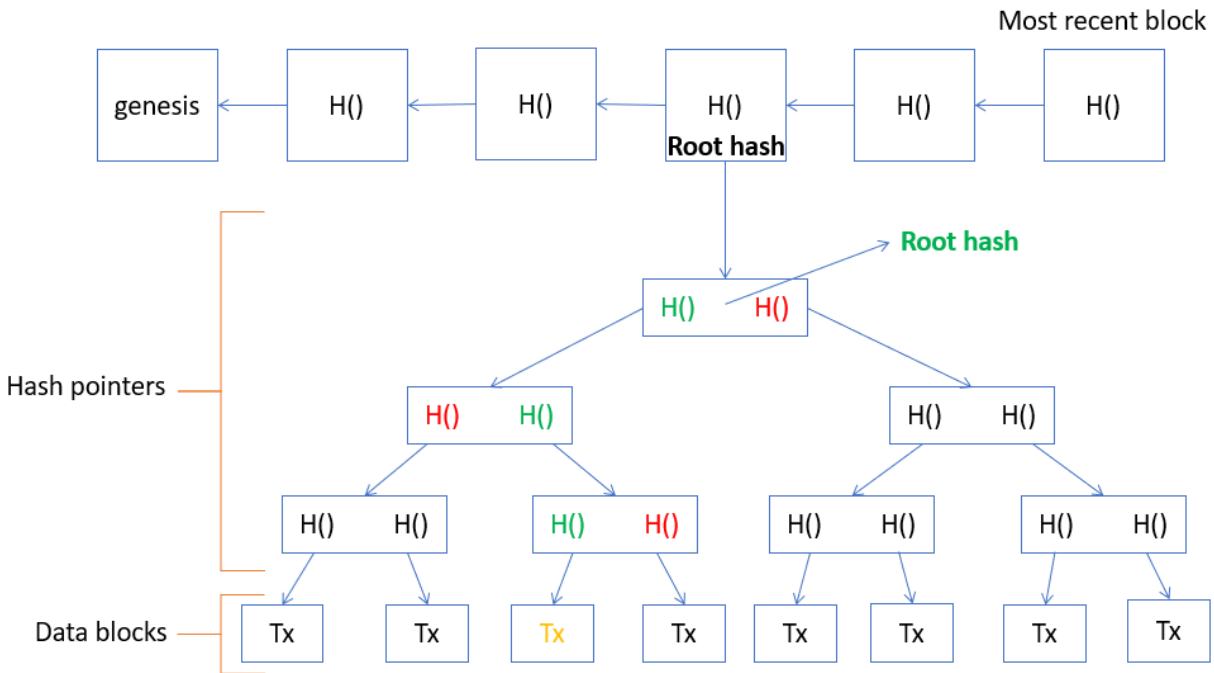
Blocks in Bitcoin are connected by hash pointers, and all the transaction information contained in each block is stored in the form of a Merkle tree. Each block is divided into a block header and a block body. The root hash value is stored in the block header, but the specific transaction information is not. The specific transaction information is stored in the block body.

Bitcoin nodes are divided into full nodes and light nodes. Full nodes store the entire blockchain, while light nodes, such as Bitcoin wallet apps on mobile phones, store only block headers.

The question, then, is: How can a light node tell whether a particular transaction exists on the blockchain? This requires the use of a Merkle proof. The light node knows only the root hash value, and it now needs to verify whether the transaction marked in yellow in Figure 5 exists. The light node sends a request to a full node, asking it to provide a Merkle proof. The full node

Fig 5

Some people may wonder: Can someone tamper with the hash values provided by the full node and try to make the calculated root hash match the stored root hash? This is theoretically feasible, but it is difficult in practice because of the collision-resistance property. It is very difficult to artificially create hash collisions. Therefore, a Merkle proof is a reliable way to prove whether a transaction actually exists in the blockchain. This verification process is also called a proof of membership or inclusion, and its time complexity is $O(\log n)$.



Can it prove that a particular transaction does not exist? Yes, but doing so is troublesome and inefficient. The least efficient method is to send the entire tree to the light node, but the time complexity of this method is $O(n)$. Is there a more efficient way? The answer is no, unless we sort the leaf nodes of the Merkle tree. We can then reduce the time complexity to $O(\log n)$. Bitcoin does not implement a sorted Merkle tree.

Bitcoin Protocol

Digital Currency System Design

Let's put concerns about centralization aside for now and think about how to design and issue a digital currency from a central bank's perspective. A central bank is a centralized institution. It can use the concept of a public-private key pair to issue digital currency. Suppose the central bank issues a digital file representing \$100 (Figure 6) and digitally signs it using its private key. This document is then distributed to the public. If A buys something worth \$100, A transfers this \$100 digital file to the seller. After receiving the file, the seller can use the central bank's public key to verify that the file was indeed issued by the central bank. Now, the question is: Does this design work?



\$100
(Signed by central bank's
private key)

Fig 6

The answer is no, although people cannot tamper with the contents of the file.

Why can't we change the contents of the file—for example, by turning \$100 into \$1,000? This is because the \$100 digital currency file must first go through a hash function to generate a digest. The central bank then uses its private key to sign the digest rather than the original file. The reason this design does not work is that people can copy and paste the file. The copied file also has a verifiable signature from the central bank, creating the risk of a **double-spending attack**.

Therefore, a successful digital currency system must be able to solve the double-spending problem. Now, let's try to improve our design. This time, we add a serial number to each \$100 file, just like the serial number on a real paper bill. The central bank then creates a database to record the ownership of each \$100 file (Figure 7).

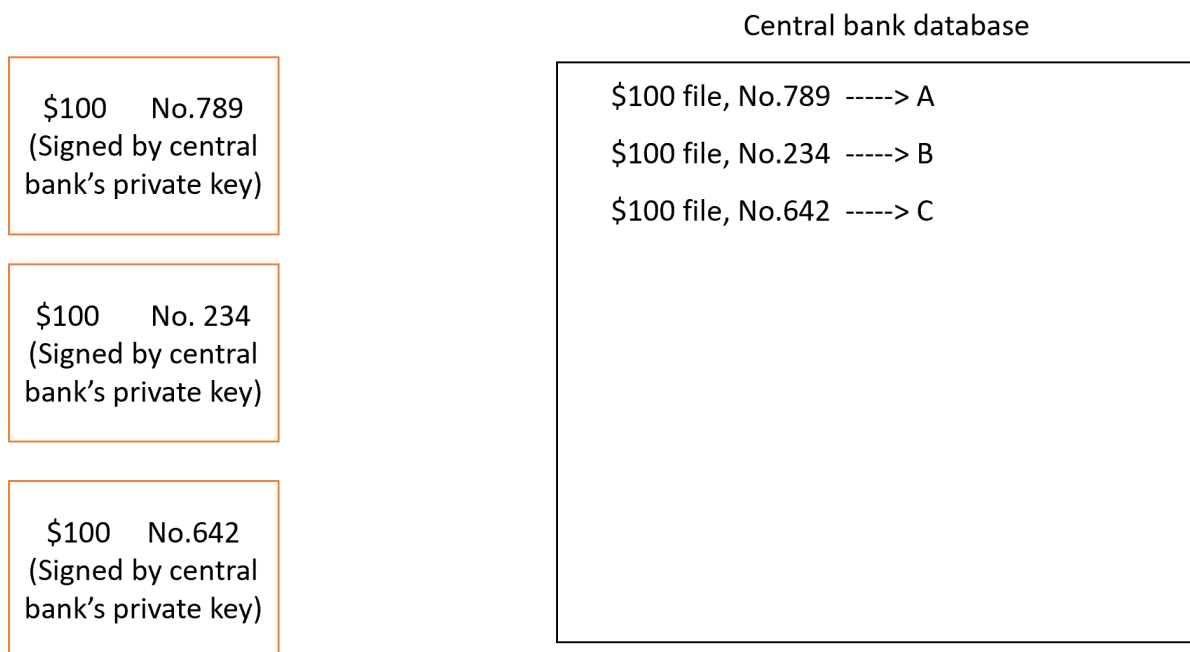


Fig 7

When money is spent, not only must the signature be verified, but the database must also be checked to determine whether the \$100 in digital currency being spent actually belongs to that person. After the transaction is confirmed, the database must be updated. If the person attempts to spend the same \$100 in digital currency again, we can check the central bank's database and easily determine that the \$100 file has already been spent and is no longer in that person's possession. This updated design prevents people from double-spending. Is there any problem with this design? Not really. This design is feasible, but it is completely centralized. Now, let's consider how Bitcoin transforms this design into a decentralized system.

The two major problems that a decentralized digital currency must solve are as follows:

1. Who has the power to issue the digital currency? How much currency should be issued, and how often?
2. How can double-spending attacks be prevented?

Bitcoin solves the first problem through **mining**, which will be discussed in later chapters. Let's discuss the second question first: How does Bitcoin solve the double-spending problem? Bitcoin's solution is similar to the centralized design mentioned above. It also requires a data structure to keep a record of who has spent the bitcoin. The difference is that this data structure is not maintained by a centralized organization, but by all users. This data structure is called the blockchain.

The diagram illustrates a sequence of four blocks in a Bitcoin block chain, connected by blue arrows pointing from right to left. Each block is a blue-outlined rectangle containing transaction details and a signature. Orange curved arrows highlight the flow of data from a transaction in one block to the input of a transaction in the previous block.

- Block 1 (Leftmost):** Contains the transaction "Coinbase TX----> A(10)".
- Block 2:** Contains two transactions: "A ----> B (5)" and "A ----> C (5)", followed by "A's sig".
- Block 3:** Contains two transactions: "B ----> C (2)" and "B ----> D (3)", followed by "B's sig".
- Block 4 (Rightmost):** Contains the transaction "C ----> E(7)" followed by "C's sig".

Orange curved arrows show the following connections:

- From "A ----> B (5)" in Block 2 to "Coinbase TX----> A(10)" in Block 1.
- From "A ----> C (5)" in Block 2 to "A ----> B (5)" in Block 2.
- From "B ----> C (2)" in Block 3 to "A ----> C (5)" in Block 2.
- From "C ----> E(7)" in Block 4 to "B ----> C (2)" in Block 3.

Fig 8

transaction contains two parts: **inputs** and **outputs**. The inputs are the 5 bitcoins and provide the sender's public key. The outputs are the 5 bitcoins received from the sender and provide the receiver's public key.

Now, let's look at the transaction from A to B. A is the sender and B is the receiver. A has 5 bitcoins and B has 5 bitcoins. A is sending 5 bitcoins to B. Now, the source is a little more complex. The 5 bitcoins came from A.

Blockchain. In this blockchain, the source and the other indicates the source and the other.

What is the purpose of the hash pointers that indicate the source? They are used to prevent double-spending. For example, if B wants to spend the 5 bitcoins obtained from A again, B would place the transaction information in the fifth block and sign the transaction (Figure 9). The hash pointer would point to the transaction from A to B in the second block.

After receiving the fifth block, the other nodes begin to verify it by starting with the fifth block and checking backward. When they reach the third block, B's double-spending attempt fails because the hash pointers of the transactions in the third block also point to the transaction in which A transferred the bitcoins to B in the second block. This means that the 5 bitcoins B wants to spend again in the fifth block have already been spent by B in the third block. Therefore, this double-spending attack cannot succeed.

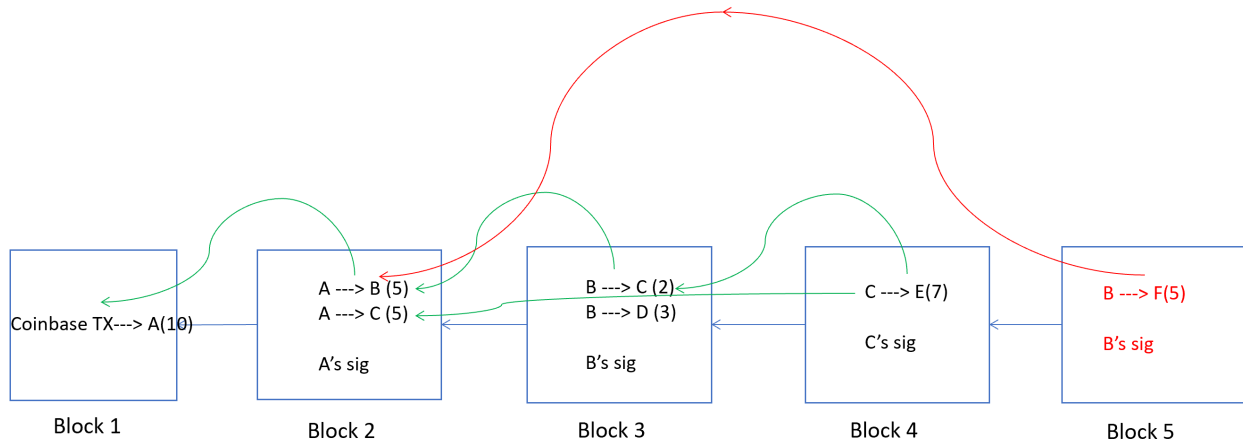


Fig 9

Let's think again about the transaction in which A transfers bitcoins to B. This transaction requires two pieces of information: A's signature and B's address. B's address is the hash of B's public key, and it is similar to B's bank account number. How did A obtain B's address in advance? Just as in real life, the banking system will not help you find someone's account number, and the Bitcoin system will not help you find someone's address. You must either ask the other party for their address or obtain it through a third party. For example, when selling something, a seller may post their address on a trading platform.

What information does B need to know about A? B needs to know A's public key because it represents A's identity. More importantly, all the other nodes also need to know A's public key. Why? Because they need to use A's public key to verify A's signature. The private key signs, and the public key verifies. Each node must independently verify A's signature because some nodes may be malicious and cannot be trusted. The question now is: How do the nodes obtain A's public key? It is actually very simple. Do you remember the information that the inputs and outputs of each transaction must contain? The input section must specify the source of the bitcoins and provide the sender's public key, while the output section must provide the hash of the receiver's public key.

Is this safe? How can you trust that the public key in the input actually belongs to A? What if a malicious node, B', replaces A's public key with its own and pretends to be A? The output of the initial coinbase transaction contained the hash of A's public key. Therefore, when A initiates the transfer to B and provides their public key, every node can trace the source back to the initial coinbase transaction. The nodes can then find the hash of A's public key recorded at that time and check whether it matches the public key provided in the input. If B' pretends to be A and places B's public key in the input field, the hash values will be different.

To reiterate, a message sent to a receiver is encrypted using the receiver's public key, and the receiver uses their own private key to decrypt the message after receiving it. When the receiver replies, they encrypt the message using the sender's public key. After receiving the message, the sender decrypts it using their own private key.

The verification process described above is implemented through scripts in the Bitcoin system. The input and output of each transaction are scripts. Verification involves combining the input script of the current transaction with the output script of the transaction that provides the source of the bitcoins to determine whether they can be executed without error. The input section must specify the source of the bitcoins and provide the sender's public key, while the output section must provide the hash of the receiver's public key. Later, we will discuss Bitcoin Script in detail.

Figure 9 above is simplified because it shows only one transaction in each block. In reality, each block contains many transactions, which together form a Merkle tree. As mentioned earlier, each block consists of a block header and a block body (Figure 10). The block header includes the version of the Bitcoin protocol currently being used, the hash pointer to the previous block, the root hash value of the Merkle tree, and two fields related to mining. One is the mining difficulty target, and the other is the random-number field called the nonce. nBits is the encoded value of the target.

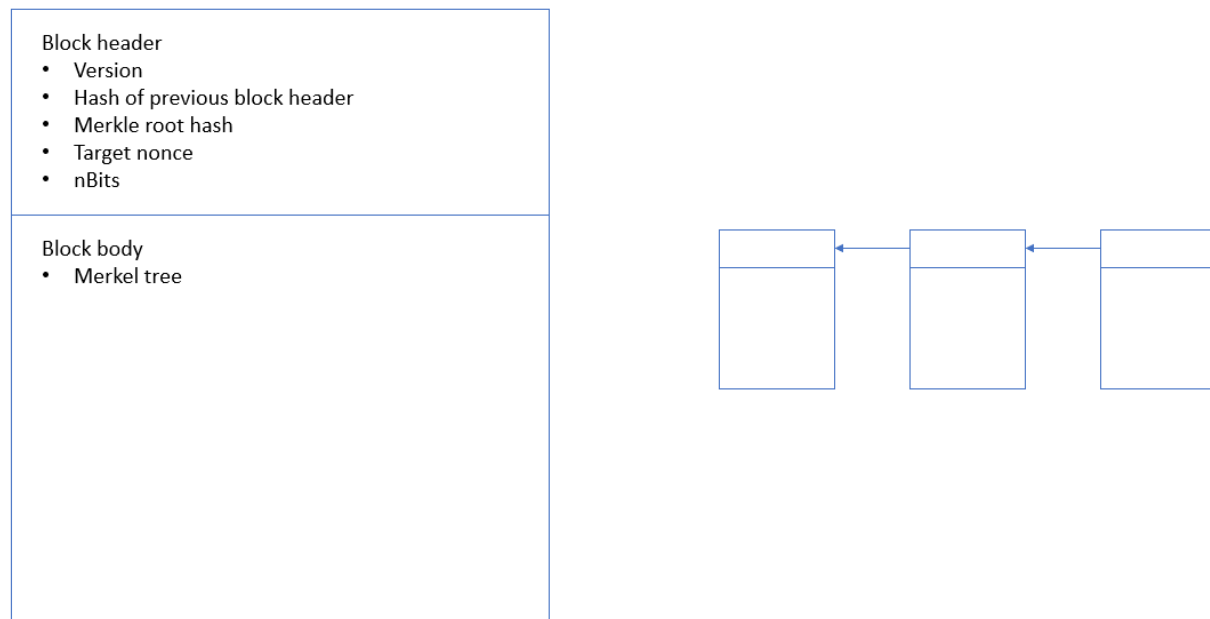


Fig 10

Note that the hash pointer to the previous block is actually calculated based on the previous block's header, rather than the entire block. Therefore, the blockchain is actually connected through a series of block headers. The block body is used to store transactions, while the root hash value of the Merkle tree contained in the block header ensures that the information in the block body cannot be tampered with.

As mentioned earlier, there are two types of nodes: full nodes and light nodes. A full node stores the entire blockchain and verifies transaction information. It is also known as a fully validating node. A light node stores only block header information. Generally speaking, a light

node cannot independently verify the validity of a transaction. For example, it cannot determine whether a transaction involves double-spending. In fact, most of the nodes in the system are light nodes. These nodes do not actively participate in constructing and maintaining the blockchain; they only retrieve information from the blockchain for verification purposes.

How exactly is the blockchain built, and how do we decide what information should be written into the next block? Bitcoin is essentially a decentralized ledger. Each node cannot simply keep its own separate ledger. We must achieve **distributed consensus** among all the nodes.

A simple example of distributed consensus is a distributed hash table. What does this mean? Multiple computers in the system jointly maintain a global hash table. These computers must reach a consensus on the key-value pairs stored in the hash table. This means that if computer A reads the key "001" and obtains "Xuan" as its value, computer B should also obtain "Xuan" when it reads the key "001."

This involves concepts from **distributed systems**, including several FLP impossibility results. For example, in an asynchronous system, consensus cannot be reached even if only one member is faulty. Another famous result is the CAP theorem, which states that only two of the following three properties can be satisfied at the same time:

- Consistency
- Availability
- Partition tolerance

A well-known protocol for achieving distributed consensus is Paxos. This protocol can achieve consistency, but it may be unable to do so in certain situations. The theories above have little to do with the practical operation of Bitcoin, so we will end the discussion here.

Bitcoin consensus

One of the concerns that the Bitcoin network must address is the presence of malicious nodes. How should we design a consensus protocol to ensure the system's safety? Assuming that most nodes are honest and malicious nodes are in the minority, can we reach consensus by voting based on the number of accounts?

Let's assume that a particular node proposes a candidate block and everyone votes on it. Any voting system must first confirm membership, and only approved members can participate in the voting process. However, creating an account is very easy in the Bitcoin system. People can quickly generate thousands of public-private key pairs. If we generate more than half of all the accounts, we will naturally be able to control the Bitcoin system. This type of attack is called a **Sybil attack**. Therefore, this account-based voting system will not work.

How does the Bitcoin system solve this problem? It also uses voting, but instead of account-based voting, Bitcoin uses computing power to vote. Each node can form a candidate block locally and include transactions that it considers valid. The node then begins trying different

nonce values to determine which one produces a hash value that falls within the specified target range (Figure 11). If a node finds a matching nonce, it obtains the right to append its candidate block to the blockchain. In this block, the node can include a coinbase transaction that pays the block reward to itself.

$$H(\text{Block header}) \leq \text{target}$$

Nonce: 4 bytes

Fig 11

After other nodes receive this block, they begin to verify it. There is a field in the block header called nBits, which is the encoded value of the target range. The verification process includes checking whether the nBits field meets the difficulty requirements of the Bitcoin protocol, whether the hashed nonce falls within the target range, and whether each transaction in the block body is valid. This includes checking whether the signatures and the sources of the bitcoins are valid. If anything is invalid, the block will not be accepted by the other nodes. After a node receives a block, how does it know where to append it? The answer is simple: it looks at the hash pointer that points to the previous block.

If a new block passes all these checks, will it necessarily be accepted and added to the blockchain? Suppose a node obtains the right to append a block and then publishes it. However, this block needs to be appended in the middle of the blockchain rather than at the end. If its block header and block body are both valid, should we accept it (Figure 12)?

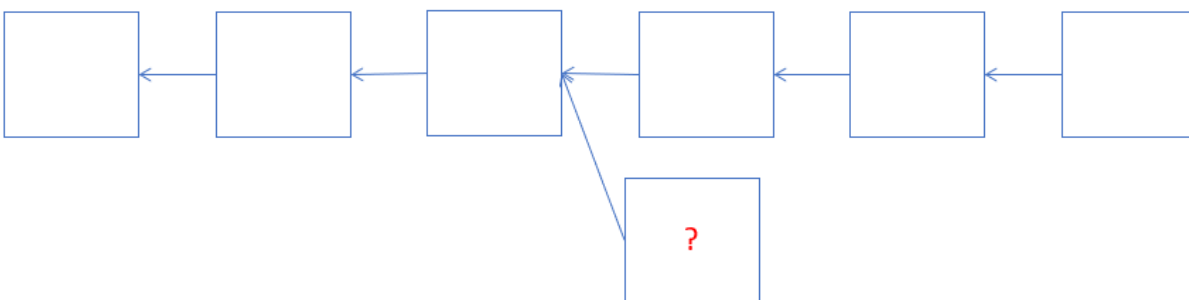


Fig 12

To answer the questions above, let's first look at the following example (Figure 13). There are three transactions on the blockchain: C transfers bitcoins to A; A transfers bitcoins to B; and A transfers bitcoins to another account, A', which also belongs to A. However, this last transaction occurs on a side chain.

Remember that when we determine whether double-spending has occurred, we need to consider whether the two transactions occurred on the same chain. In this case, the transfer from A to A' does not constitute double-spending because the transfers from A to B and from A to A' occur on different chains.

For example, to check the transfer from A to A', we need to trace the side chain all the way back to the transaction from C to A. Along this path, A has not transferred the bitcoins to anyone else, so the transfer from A to A' is valid. However, although the transfer from A to A' is valid, this transaction is not on the **longest valid chain**. The Bitcoin system stipulates that an accepted block must extend the longest valid chain. Therefore, the transaction from A to A' is valid but has no effect.

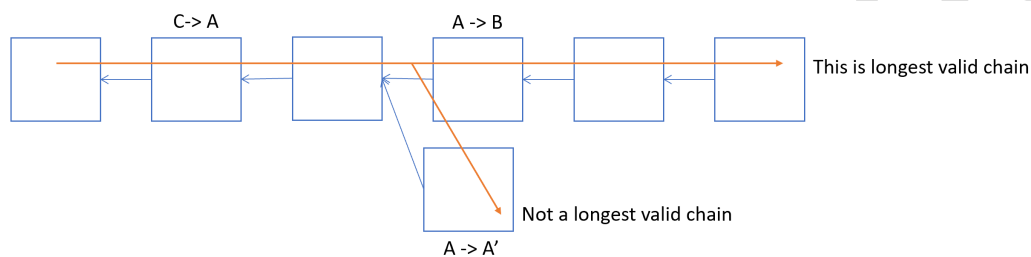


Fig 13

The case above is an example of a **forking attack**, which attempts to reverse a transaction that has already occurred on the longest valid chain by inserting a valid block in the middle of the blockchain. However, the blockchain may also fork during normal operation. For example, two nodes may obtain the right to append the next block at the same time (Figure 14), and both may broadcast their own blocks. In this case, there will be two chains of equal length. Both chains are the longest valid chains, so which one should we accept?

Depending on their locations in the Bitcoin network, some nodes may receive the upper block first, while others may receive the lower block first. Therefore, the two valid chains of equal length may coexist for a period of time. Eventually, however, one of them will win (Figure 15).

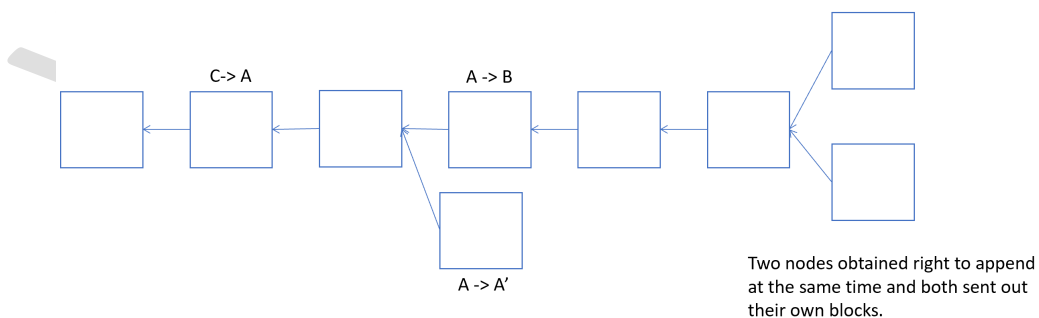


Fig 14

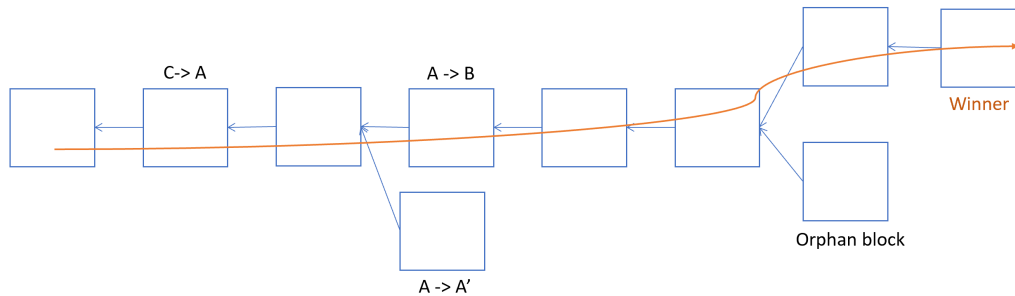


Fig 15

Note that the Bitcoin protocol uses implicit consent. This means that if a node continues to append blocks after the block you broadcast, the node has accepted your block. The winning chain becomes the longest valid chain, while the losing block is discarded as an orphan block.

Why do these nodes spend time and effort obtaining the right to append the next block? As mentioned earlier, the node that obtains the right to append a block has the power to decide which transactions will be included in it. However, this is not the main reason, since we hope that all valid transactions will be included in the block.

Bitcoin rewards a node that obtains the right to append a block by giving it a **block reward**. This reward takes the form of a special coinbase transaction that the node can include in the block. The coinbase transaction is the only way to issue new bitcoins in the Bitcoin system. In Bitcoin's early days, each node that obtained the right to append a block could receive a reward of 50 bitcoins. However, the Bitcoin protocol stipulates that this reward must be halved after every 210,000 blocks (Figure 16).

50 BTC -> 25 BTC -> 12.5 BTC

Fig 16

Note that in the case above, the node obtained the right to append a block but lost the competition to become part of the longest valid chain. Its published block would therefore become an orphan block, making the block reward in that block useless because the other nodes would not accept it. Other nodes accept only the longest valid chain.

What exactly must the Bitcoin system reach a consensus on? In fact, it must reach a consensus on Bitcoin's longest valid chain. This chain is the "ledger." Obtaining the right to append a block requires solving a puzzle, and solving the puzzle requires computing power. The greater the computing power, the greater the probability of obtaining the right to append a block. This so-called computing power is actually the number of possible nonce values that can be tried per second, also known as the **hash rate**.

How does this design prevent Sybil attacks? The right to append a block depends on the amount of computing power rather than the number of accounts. Therefore, even if a person creates hundreds of accounts, doing so will not increase that person's hash rate. The essence of so-called mining is simply solving a puzzle by trying different nonce values.

xuanf@usc.edu

Bitcoin System Implementation

UTXO

The Bitcoin system uses a **transaction-based ledger**. It does not specifically record how many bitcoins each account has. If we want to know how many bitcoins account A has, we must calculate the amount based on A's past transaction information.

Full nodes in the Bitcoin system must maintain a data structure called the **UTXO** set, where UTXO stands for "unspent transaction output." It contains transaction outputs that have not yet been spent. There are many transactions on the Bitcoin blockchain. Some of their outputs have been spent, while others have not. The set of all transaction outputs that have not yet been spent is called the UTXO set.

A transaction may have multiple outputs. For example, A transfers 5 BTC to B and 3 BTC to C. After receiving the BTC, B spends it. C receives the BTC but does not spend it. Therefore, only the transaction output from A to C remains in the UTXO set (Figure 17). Each element in the UTXO set must include the hash value of the transaction to which the output belongs and the output's index within that transaction.

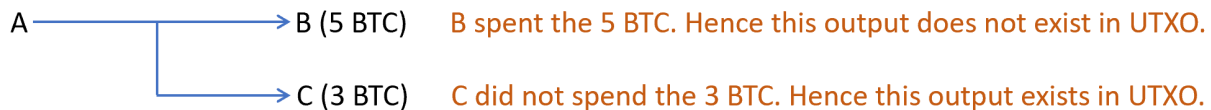


Fig 17

So, what is the UTXO set used for? It is used to detect double-spending. For any new transaction, you need to check the UTXO set first because the bitcoins you want to spend must be in it. All full nodes need to maintain the UTXO data structure in memory so that they can quickly detect double-spending.

Each transaction can have multiple inputs and multiple outputs, and the total number of bitcoins in all the inputs must be equal to the total number of bitcoins in all the outputs. The transaction shown in the figure above is an example of a transaction with a single input and multiple outputs. Note that multiple inputs do not necessarily come from the same address; they can come from different addresses. Therefore, a transaction may require multiple signatures because each input address must provide a corresponding signature.

The **total inputs** in each transaction are usually slightly greater than the **total outputs**. The difference is the **transaction fee**, which is paid to the node that obtains the right to append the next block. As mentioned earlier, nodes compete for the right to append blocks in order to obtain coinbase transaction rewards. The fee earned by including other people's transactions is also a reward that encourages the node with the right to append a block to record those transactions. Without transaction fees, the node would have no incentive to help others include their transactions in the block.

On average, the Bitcoin system generates a new block every 10 minutes. After every 210,000 blocks, the block reward is halved ($210,000 \times 10$ minutes means that the reward is halved approximately every four years).

Bitcoin uses a **transaction-based ledger**, while another design, called an **account-based ledger**, is used by Ethereum. An account-based ledger records how many coins each account has. Bitcoin's transaction-based ledger requires every transaction to be traceable. We must be able to identify the source of the bitcoins, including which transaction they came from and which output number they correspond to. Because Ethereum uses an account-based ledger, there is no need to specify the source of the coins.

Let's take a look at a real block (Figure 18).

Block #529709

Summary	
Number Of Transactions	686
Output Total	4,220.46616378 BTC
Estimated Transaction Volume	651.93844862 BTC
Transaction Fees	0.12458867 BTC
Height	529709 (Main Chain)
Timestamp	2018-06-29 06:17:26
Received Time	2018-06-29 06:17:26
Relayed By	BTC.com
Difficulty	5,077,499,034,879.02
Bits	389508950
Size	333.53 kB
Weight	1160.618 kWU
Version	0x20000000
Nonce	3897564446
Block Reward	12.5 BTC

Hashes	
Hash	000000000000000000030f1531dbfa069037188c5048b23f0c979ce8728de8c5
Previous Block	000000000000000000000841c59a4679d6e707152f24f5b195b66b47397d65175e
Next Block(s)	
Merkle Root	6f73d36264f05e0c55c4703f85e85628a29231e9673bce3f302bfe76dc2b378

source: blockchain.info

Fig 18

“Height” is the block’s serial number.

“Difficulty” refers to the mining difficulty. It is adjusted every 2,016 blocks to keep the average block time at approximately 10 minutes.

“Hash” is the hash value of the block header.

“Previous block” is the hash value of the previous block’s header.

“Merkle root” is the root hash value of the Merkle tree formed by the transactions contained in the block.

The following is the structure of the block header (Figure 19).

Block header

```
13  /** Nodes collect new transactions into a block, hash them into a hash tree,
14  *   and scan through nonce values to make the block's hash satisfy proof-of-work
15  *   requirements. When they solve the proof-of-work, they broadcast the block
16  *   to everyone and the block is added to the block chain. The first transaction
17  *   in the block is a special one that creates a new coin owned by the creator
18  *   of the block.
19  */
20  class CBlockHeader
21  {
22  public:
23      // header
24      int32_t nVersion;
25      uint256 hashPrevBlock;
26      uint256 hashMerkleRoot;
27      uint32_t nTime;
28      uint32_t nBits;
29       uint32_t nNonce;
```

source: bitcoin/src/primitives/block.h

Fig 19

As shown here, the data type of the nonce field is uint32_t (4 bytes). This means that there are at most 2^{32} possible values. As the mining difficulty increases, adjusting the nonce field alone may not produce a hash value that meets the requirements. So, which of these fields can be modified (Figure 20)?

Block headers are serialized in the 80-byte format described below and then hashed as part of Bitcoin's proof-of-work algorithm, making the serialized header format part of the consensus rules.

Bytes	Name	Data Type	Description
4	version	int32_t	The block version number indicates which set of block validation rules to follow. See the list of block versions below.
32	previous block header hash	char[32]	A SHA256(SHA256()) hash in internal byte order of the previous block's header. This ensures no previous block can be changed without also changing this block's header.
32	merkle root hash	char[32]	A SHA256(SHA256()) hash in internal byte order. The merkle root is derived from the hashes of all transactions included in this block, ensuring that none of those transactions can be modified without modifying the header. See the merkle trees section below.
4	time	uint32_t	The block time is a Unix epoch time when the miner started hashing the header (according to the miner). Must be strictly greater than the median time of the previous 11 blocks. Full nodes will not accept blocks with headers more than two hours in the future according to their clock.
4	nBits	uint32_t	An encoded version of the target threshold this block's header hash must be less than or equal to. See the nBits format described below.
4	nonce	uint32_t	An arbitrary number miners change to modify the header hash in order to produce a hash less than or equal to the target threshold. If all 32-bit values are tested, the time can be updated or the coinbase transaction can be changed and the merkle root updated.

The hashes are in internal byte order; the other values are all in little-endian order.
source: bitcoin.org

Fig 20

“Version” is the version number of the Bitcoin protocol currently in use. This field cannot be modified.

“Previous block header hash” is the hash value of the previous block’s header, and it cannot be modified.

“Merkle root hash” is the root hash value of the Merkle tree, and it can be changed.

“Time” is the time at which the block was generated. There is some room for adjustment because the Bitcoin system does not require the time to be precise, as long as the adjustment is minor.

“nBits” is the target value for mining. It can be modified only periodically and in accordance with the requirements of the protocol; it cannot be modified arbitrarily.

“Nonce” is the random-number field whose possible values we need to try.

When adjusting the nonce field is not enough, we can modify the Merkle root hash field. Why can the Merkle root hash be modified? As mentioned earlier, each newly generated block contains a coinbase transaction reward, which is the only way to generate new bitcoins in the

Bitcoin system (Figure 21). The bitcoins obtained through this transaction have no inputs because they are generated by the Bitcoin system itself. This transaction contains a field called the **coinbase field**. You can write any content in it; no one cares about or checks its contents. If we change the contents of the coinbase field, it will affect the root hash value in the block header.

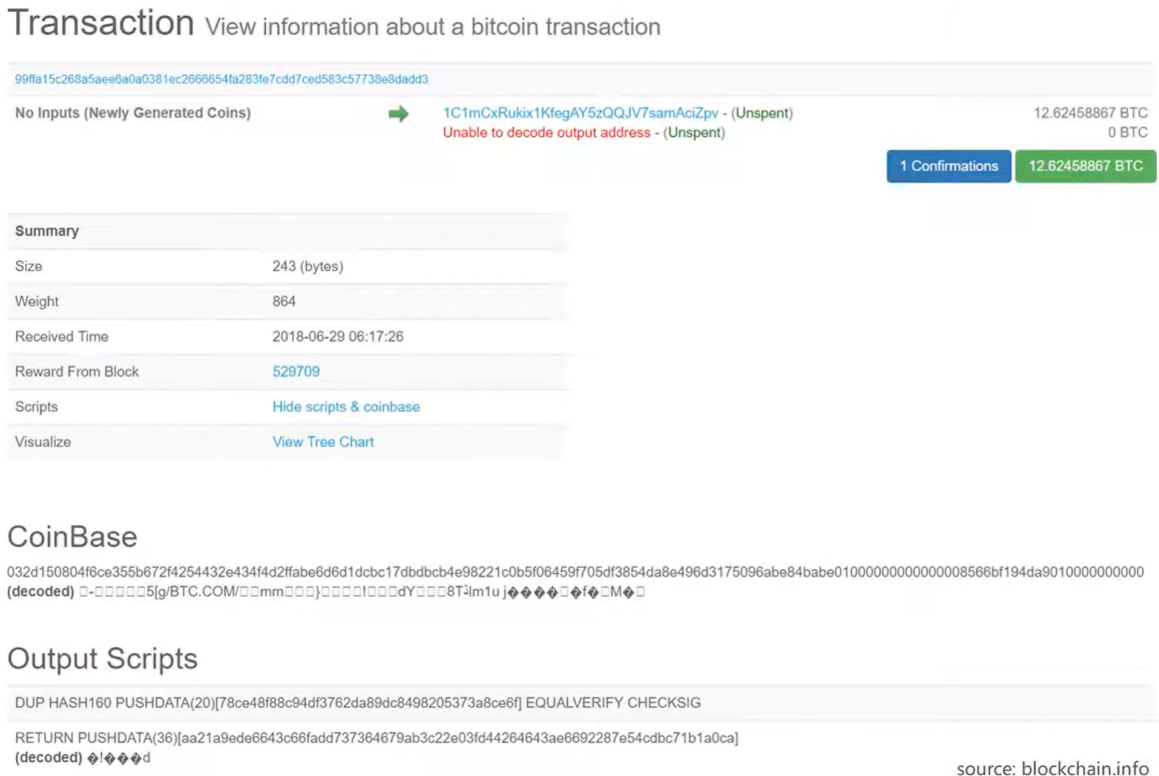
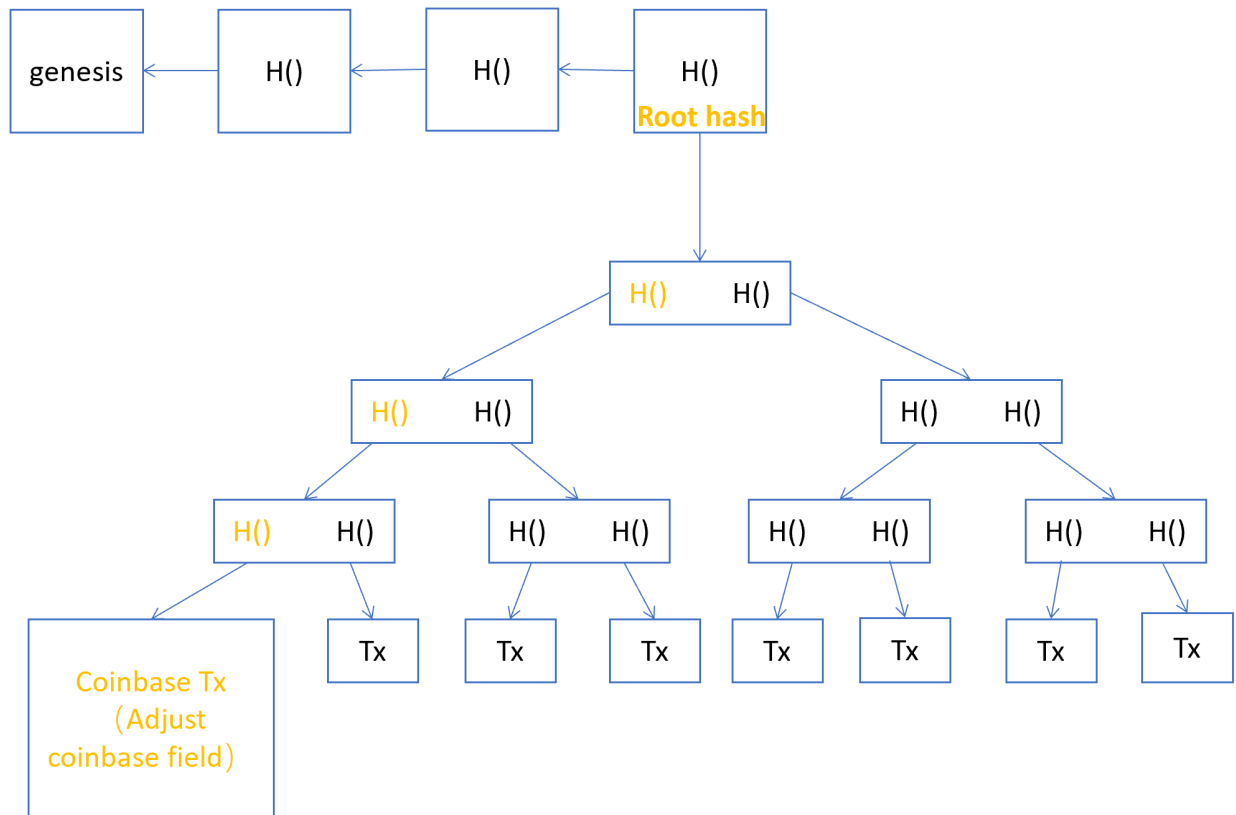


Fig 21

After we change the coinbase field, the hash value of the transaction also changes. This change then propagates upward through the Merkle tree, eventually causing the root hash value to change (Figure 22). Therefore, we can use the coinbase field as an extra nonce field.

The original nonce field is only 4 bytes in size, providing 2^{32} possible values. If this is not enough, we can use additional bytes from the coinbase field. For example, we can use the first 8 bytes of the coinbase field, providing 2^{64} additional possibilities. In this way, we can obtain a total search space of 2^{96} .

Therefore, mining actually involves two nested loops. The outer loop adjusts the extra nonce in the coinbase field to calculate the root hash value in the block header, while the inner loop adjusts the original nonce field.



Let's look at a transaction example (Figure 23). This transaction has two inputs and two outputs. Although the inputs—the items in the red box—are labeled as outputs here, do not be confused. This means that the inputs of the current transaction are outputs from previous transactions.

In the output section, you can see that all the outputs are unspent, so they need to be recorded in the UTXO set. You can also see that there is a difference between the total input and output amounts. This difference is the transaction fee.

At the bottom of the figure, you can see that the inputs and outputs are specified using scripts. Bitcoin verifies the validity of a transaction by pairing an input script with an output script and then executing them together.

Note that the two scripts shown here are not paired with each other because they belong to the same transaction. Instead, the input script of the current transaction is paired with the output script of the previous transaction that provided the source of the bitcoins. If the two scripts can be executed together without errors, the transaction is valid.

Transaction View information about a bitcoin transaction

3f9d34dc733a614f14a930a470241e0d99ea6965f99f11a6f85396a6645f	
14BHEP2sNRUj5j5SogBjysza87psR2Uv (0.00408688 BTC - Output)	1Kqj3Qiqbd1ah9G0mm8comNDKgxVvLVF - (Unspent) 0.0396 BTC
1K7q5sqzbTTtd06MtZDhb9E4jJCShiUcR (0.04758063 BTC - Output)	17Eu6pyMUJZHoaEKq3u8k2D3zdw9QYRT - (Unspent) 0.01019031 BTC
1 Confirmations 0.04979031 BTC	
Summary	
Size	373 (bytes)
Weight	1492
Received Time	2018-06-29 06:13:26
Lock Time	Block: 529708
Included In Blocks	529709 (2018-06-29 06:17:26 + 4 minutes)
Confirmations	1 Confirmations
Visualize	View Tree Chart
Inputs and Outputs	
Total Input	0.05166751 BTC
Total Output	0.04979031 BTC
Fees	0.0018772 BTC
Fee per byte	503.271 sat/B
Fee per weight unit	125.818 sat/WU
Estimated BTC Transacted	0.0396 BTC
Scripts	Hide scripts & coinbase

Input Scripts

ScriptSig: PUSHDATA(71)
[304402200e902feaf8e49ea467bbc3d9034bdf33fedfbfb662e75e8822372a3c0871e102204176d40c1781ca6f465d47db3628e2610f53d5a54ceb24e6118296ae71df5e5201]
PUSHDATA(33)[03b339dc9b56131ad95753f6995808fcc2c686bad4f69aa0b9bc9e564315c1e515]

ScriptSig: PUSHDATA(72)
[3045022100eade6baa42d209d279033581a593340780181aa0dd8a7fd2d5b6162acd81ca4d022074bd1a62c6138505823efae9ec62d4dd16e57c454a245be25f961a6e8144930201]
PUSHDATA(33)[02ceddac2ca907c010dfea74dc3c4bc27a17dcab0a2e88993cdccc243c818279ed]

Output Scripts

DUP HASH160 PUSHDATA(20)[cea9fb4638839ad9ebc9c8382dd03e2666aaeb65] EQUALVERIFY CHECKSIG

DUP HASH160 PUSHDATA(20)[4471a74ad7287cbbf6e6d1c092b22b55c886c1de] EQUALVERIFY CHECKSIG

source: [blockchain.info](#)

Fig 23

Bitcoin Mining

Memoryless

Let's discuss the probability analysis of the mining process. Mining involves constantly trying different nonce values, and each attempt can be regarded as a Bernoulli trial.

A Bernoulli trial is a random experiment with only two possible outcomes. For example, consider tossing a coin. There are only two possible outcomes. If the coin is unbalanced, the probabilities of landing on heads and tails will be different. The probability of one outcome is P , while the probability of the other is $1-P$.

When we mine BTC, the probabilities of success and failure are very different. Every time we try a nonce, the probability of failure is much greater than the probability of success. If we conduct many Bernoulli trials, these trials form a Bernoulli process.

A Bernoulli process is a series of independent Bernoulli trials. One property of a Bernoulli process is memoryless, which means that the results of past trials do not affect the results of future trials. For example, if we toss a coin 100 times, the result of one toss will not affect the result of the next toss.

When there are many trials but only a few successes, we can use a Poisson process as an approximation. In BTC mining, block times can be seen as following an exponential distribution (Figure 24).

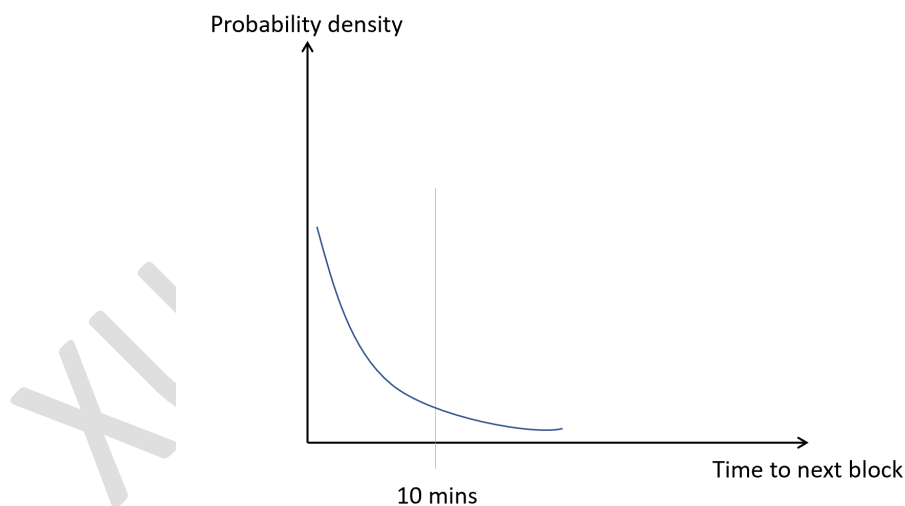


Fig 24

The figure above shows the block time of the entire system, rather than that of a specific miner. The average block time of the entire system is approximately 10 minutes. This average time is determined by the Bitcoin protocol. By regularly adjusting the mining difficulty, the system maintains an average block time of 10 minutes.

For a specific miner, the probability of mining a block depends on the miner's percentage of the system's total computing power. For example, if your computing power represents 1% of the total computing power, then, on average, you will mine one out of every 100 blocks generated. In other words, it will take you an average of 1,000 minutes to mine a block.

One characteristic of the probability density curve is that if you cut it at any point, the remaining part of the curve has the same shape as the original curve. This means that the distribution is **memoryless**. So, what does this mean for us?

For example, suppose we have already been mining BTC for 10 minutes. Since the average block time is also 10 minutes, does this mean that we are likely to mine a block soon? The answer is no. At any point during the mining process, we still have to wait an average of 10 minutes because of the memoryless property.

What is the significance of this property? It ensures that the time spent mining in the past has nothing to do with the mining time required in the future. Therefore, this property is also called **progress-free**, meaning that past progress does not affect future progress.

Now, imagine that the mining puzzle of a particular cryptocurrency is not designed to be memoryless. In that case, the longer we have been mining, the greater our chances of mining a block in the future. This creates a problem: miners with greater computing power would gain a disproportionate advantage because, over the same period, they can always perform more work than miners with less computing power.

For example, under ideal conditions, if miner A has 10 times as much computing power as miner B, the probability of A mining a block should also be 10 times that of B for the system to be fair. If the mining puzzle were not memoryless, miner A's probability of mining a block would be more than 10 times that of B. Therefore, the memoryless, or progress-free, property helps ensure the fairness of mining.

Total Supply

Let's discuss the total supply of Bitcoin. As mentioned earlier, coinbase transactions are the only way to generate new bitcoins, and block rewards are halved every four years. Therefore, the number of bitcoins generated forms a geometric series. There will be 21,000,000 bitcoins in total.

$$21k \times 50 + 21k \times 25 + 21k \times 12.5 + \dots$$

$$21k \times 50 \times \left(1 + \frac{1}{2} + \frac{1}{4} + \dots \right) = 21,000,000 \text{ BTC}$$

Solving Bitcoin mining puzzles has no practical significance by itself. Bitcoin is becoming increasingly difficult to mine, and the system is designed to halve the block reward every 210,000 blocks. This scarcity is artificially created. Although solving mining puzzles has no

practical significance by itself, the design of the mining process is the cornerstone of the Bitcoin system's security.

Bitcoin is secured through mining. Mining is an effective design that uses computing power to vote. As long as most of the computing power is controlled by honest nodes, the security of the system can be guaranteed.

We know that mining rewards are halved every four years, but does this reduce people's motivation to mine? It does not, because the price of Bitcoin continues to rise. When block rewards approach zero, will people still have the motivation to mine? Yes. People can earn transaction fees by including transactions in the blocks they mine.

System Safety

Let's move on to the discussion of Bitcoin system security. Assuming that most of the computing power is controlled by honest miners, can we guarantee that all the transactions written to the blockchain are valid? Suppose honest miners control 90% of the total computing power, while malicious miners control the remaining 10%. On average, the malicious miners would mine 10 out of every 100 blocks.

What would happen if a block were mined by one of these malicious miners? Could they create a false transaction and steal someone else's coins? The answer is no. These malicious miners cannot validly sign the false transaction.

What if M still included this false transaction in a block (Figure 25)? The other honest nodes would not accept the block because it contains an invalid transaction. Therefore, the honest nodes would not mine on top of this block. Instead, they would continue mining on top of the last valid block. Thus, M cannot steal A's coins in this way.

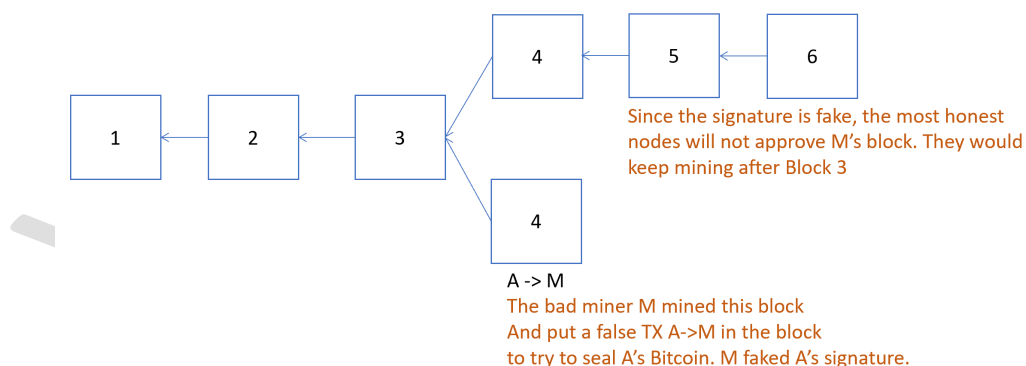


Fig 25

Now, M tries a different approach. This time, M wants to double-spend the coins (Figure 26). In Block 3, M sends the coins to A. M then mines Block 4 and includes a transaction from M to M' in an attempt to spend the same coins again. Does it work? Of course not. Each transaction has

a hash pointer that points to its source. The other nodes can easily determine that M already spent the coins in Block 3.

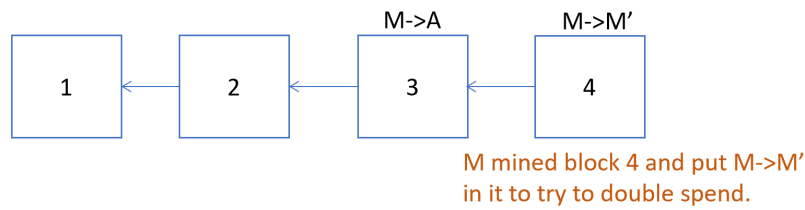


Fig 26

M does not give up. He realizes that he should append the mined block after Block 2 rather than after Block 3 (Figure 27).

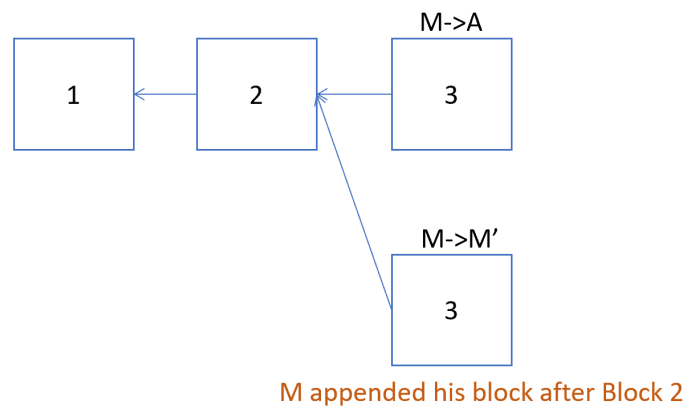


Fig 27

Note that where to append a block is determined at the beginning of the mining process, not after the block has been mined. Mining is based on the previous block. Therefore, if M wants to append a block after Block 2, he cannot mine on top of Block 3. In this case, M has actually forked the blockchain. If he manages to make his fork the longest valid chain, he will be able to double-spend the coins (Figure 28). This is the forking attack we discussed earlier.

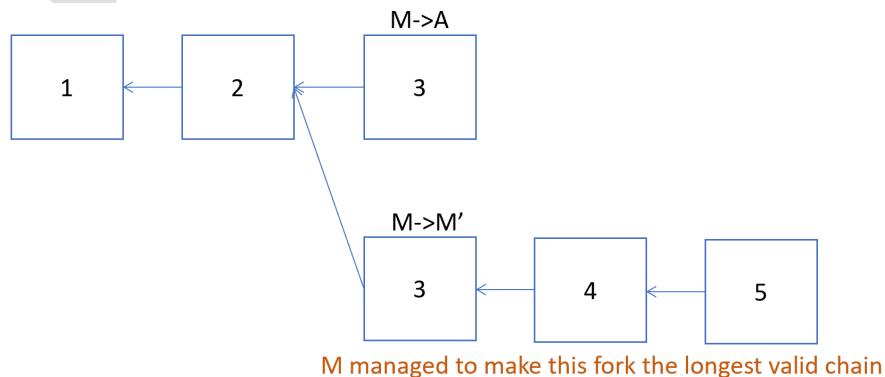


Fig 28

So, how can this attack be profitable? For example, M likes shopping online. He orders goods from A and sends bitcoins to A. After receiving the goods, M launches a forking attack and reverses the transaction from M to A. If M succeeds in making his forked chain the longest valid chain, he can obtain the goods without spending any bitcoins.

The good news is that a forking attack can be prevented. After the transaction from M to A has been written to the blockchain, A can wait for a few more confirmations (Figure 29). In other words, A waits for more blocks to be appended before confirming M's order.

At this point, it is very difficult for M to make his forked chain the longest valid chain because M must append blocks after Block 2, while the longest valid chain is already much longer than M's forked chain. Therefore, M has very little chance of making his chain longer than the current longest valid chain unless he is extremely lucky and manages to mine the next block repeatedly.

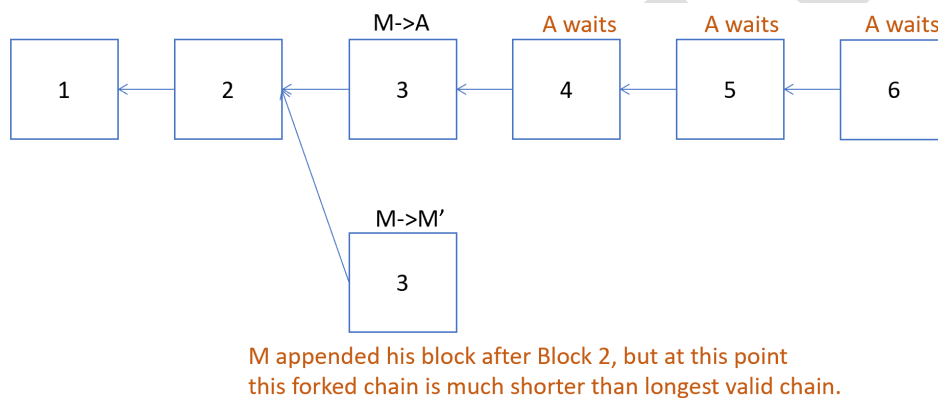


Fig 29

In the Bitcoin protocol, we wait for six confirmations, which takes approximately 60 minutes (Figure 30). When the transaction from M to A is first written to the blockchain, it has one confirmation. After that, each time another block is appended, the transaction receives one additional confirmation. After six confirmations, we can say that the transaction from M to A is “irrevocable.”

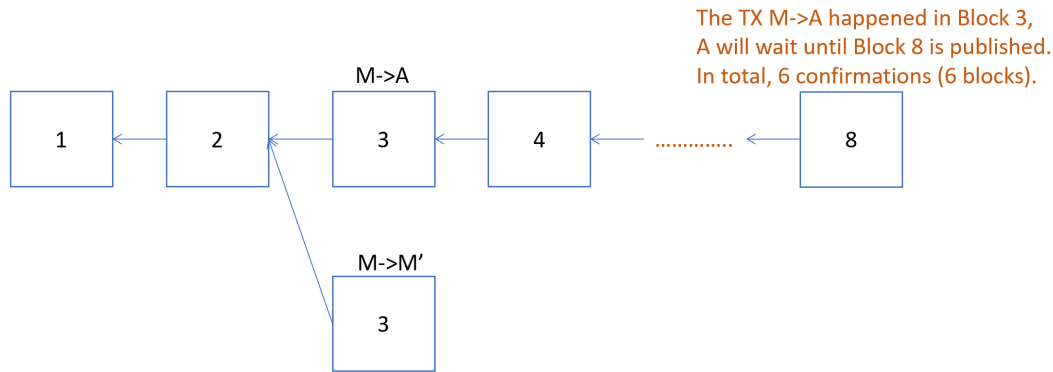


Fig 30

Please note that this so-called irrevocable ledger provides only a probabilistic guarantee. If a malicious node is extremely lucky and continuously obtains the right to append blocks, then it is theoretically possible for the node to maliciously reverse the blockchain. In other words, content that has just been written to the blockchain is relatively easy to tamper with, but after several confirmations, the probability of tampering decreases exponentially.

In fact, there is another confirmation model called **zero confirmation**. Take online shopping as an example. When we buy a laptop, we pay with Bitcoin and send a transaction to the seller. The seller then asks a full node to verify the transaction and check whether the signature and the source of the bitcoins are valid. If everything is correct, the seller approves the transaction without waiting for six confirmations.

This sounds dangerous, doesn't it? Actually, it is not. First, the Bitcoin protocol specifies that nodes will recognize only the transaction they receive first. The transaction from M to A is broadcast first, while the transaction from M to M' is broadcast later. Therefore, honest nodes will not accept the transaction from M to M'.

Second, an online shopping platform requires a certain amount of time to process and ship an order. If the seller discovers before shipment that the transaction has not been written to the longest valid chain, the seller can cancel the shipment. Therefore, problems in the Bitcoin system can sometimes be solved outside the system.

Now, let's return to the malicious miner M. After M obtains the right to append a block, what happens if M deliberately excludes some valid transactions from the block? The Bitcoin protocol does not specify which transactions a node must include in a block after obtaining the right to append it.

In fact, this is not a major problem because these valid transactions can be written to the blockchain in future blocks. There will always be honest nodes willing to publish them. Moreover, during the normal operation of the blockchain, some valid transactions may not be written to the next published block, possibly because too many transactions were submitted within a short period. The Bitcoin protocol stipulates that the size of each block cannot exceed

1 MB. Therefore, if there are too many transactions, some of them must wait to be included in future blocks.

In fact, there is another type of attack called selfish mining. Under normal circumstances, a node that obtains the right to append a block will publish it immediately. If the node does not publish it, other nodes may mine and publish their own blocks, making the unpublished block useless.

In selfish mining, however, a node obtains the right to append a block but keeps the block private and continues mining on top of it. Essentially, the node tries to extend this chain secretly. When the chain becomes long enough, the node publishes the entire hidden chain and attempts to defeat the main chain.

However, succeeding in this attack is also very difficult. After hiding the first block, the other nodes do not know that the block exists and therefore cannot mine on top of it. If the malicious node does not control at least 51% of the total computing power, the hidden chain will grow more slowly than the longest valid chain.

What if we use selfish mining not maliciously, but for normal expansion? What are the benefits? In fact, competition can be reduced to a certain extent (Figure 31). Suppose we find the next block, Block 4, but do not publish it. The other nodes will continue mining on top of Block 3. If we have greater computing power, only we will know that Block 4 exists, and only we will be able to mine on top of it. In this way, we can eliminate competition.

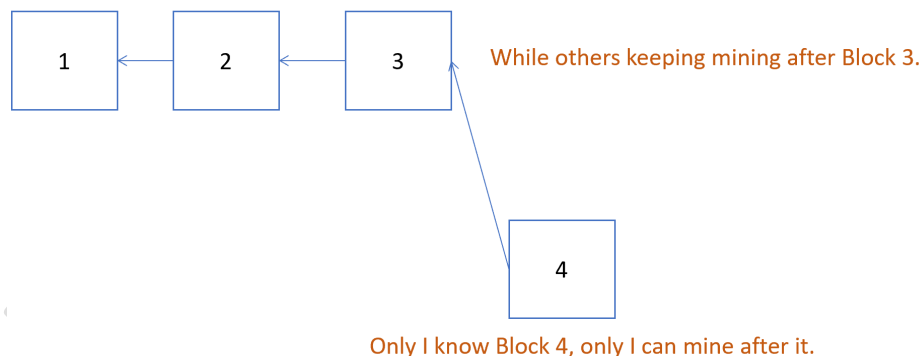


Fig 31

Of course, you need substantial computing power to achieve this. By the time the other nodes obtain their Block 4, you must have already obtained Block 5 on your own chain so that your chain is longer than the main chain (Figure 32).

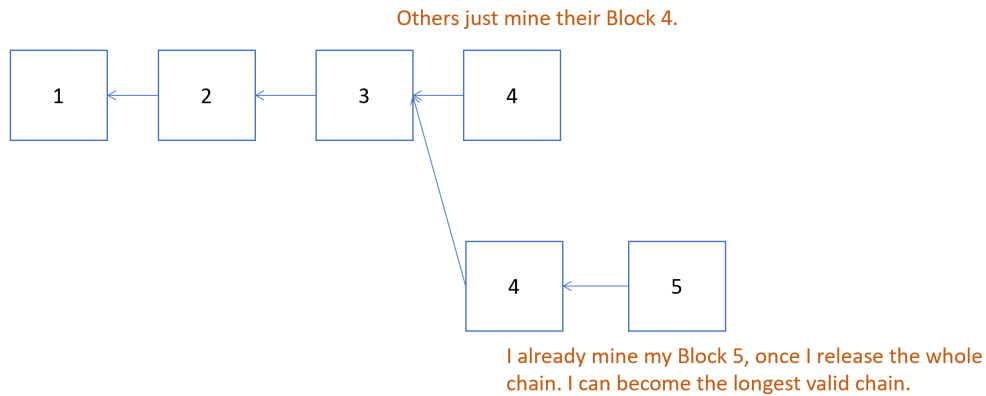


Fig 32

Mining Difficulty

As mentioned earlier, mining involves repeatedly trying different nonce values until a hash value within the target range is found. Obviously, the smaller the target range, the greater the mining difficulty. Changing the mining difficulty simply means adjusting the target range within the entire output space.

$$H(\text{block header}) \leq \text{target}$$

$$\text{difficulty} = \frac{\text{difficulty_1_target}}{\text{target}}$$

Bitcoin uses the SHA-256 hash algorithm, which generates a 256-bit hash value. Therefore, the entire output space contains 2^{256} possible values. The mining difficulty is inversely proportional to the size of the target range. The minimum mining difficulty is 1, which corresponds to a very large target range.

Why do we need to adjust the mining difficulty? As the system's computing power increases, the block generation time will become shorter and shorter if the mining difficulty remains unchanged. It may decrease from 10 minutes to less than one second. Although a shorter block time allows transactions to be written to the blockchain more quickly, it also creates some concerns.

For example, the block time may be less than one second, while it may take tens of seconds for a block to propagate throughout the Bitcoin network. Nodes that are "far away" may not receive the block quickly. Therefore, they will continue mining on top of the previous block in the chain. These nodes may then find their own blocks and publish them to the network.

When this happens, the blockchain will have many forks (Figure 33). If the block time becomes shorter and shorter, the number of forks will become larger and larger. Too many forks will make it difficult for the system to reach a consensus and will also make the system less secure.

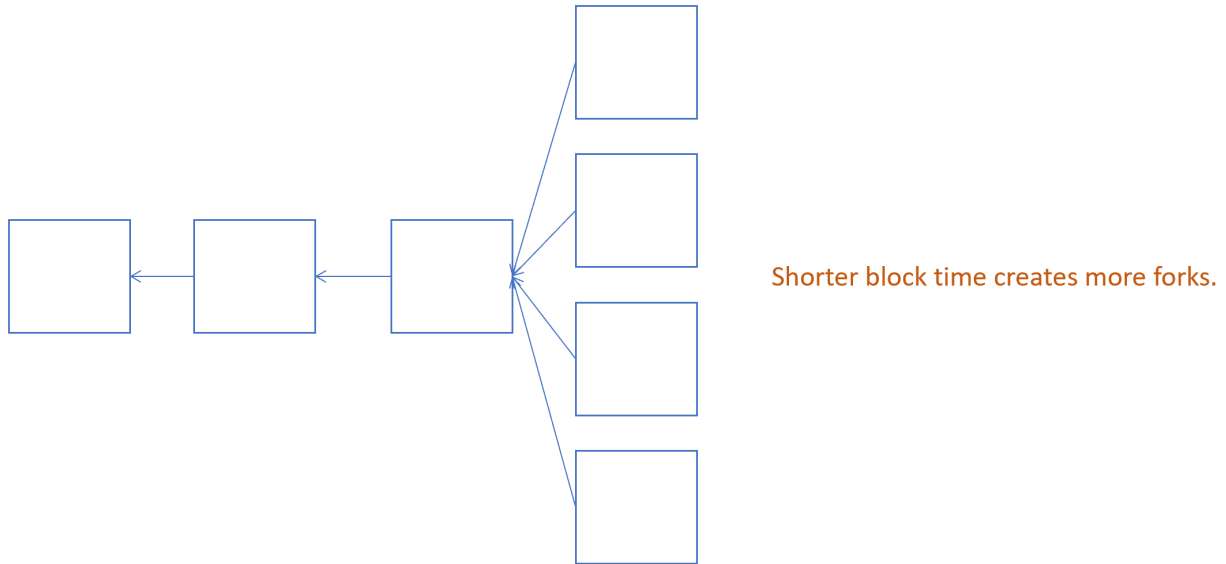


Fig 33

Why would this create a security concern? This is because the Bitcoin system assumes that most nodes are honest. Therefore, the greater the total computing power, the more secure the system is and the more expensive it becomes to launch a 51% attack.

What is a 51% attack? It occurs when malicious nodes control more than 51% of the total computing power. Suppose A transfers 1,000 bitcoins to B, and B waits for six confirmations (Figure 34). B now believes that the transaction has been successfully written to the blockchain.

At this point, the malicious user A can create a fork before the block containing the transaction from A to B in an attempt to reverse it. It is difficult for A to make this fork longer than the longest valid chain when honest nodes control most of the computing power. However, if many forks occur on the longest valid chain, the computing power will become scattered because different nodes may extend different forks.

When this happens, the malicious user A can concentrate his computing power on extending his malicious fork. A may not need 51% of the total computing power to launch the attack and may succeed with only 10%.

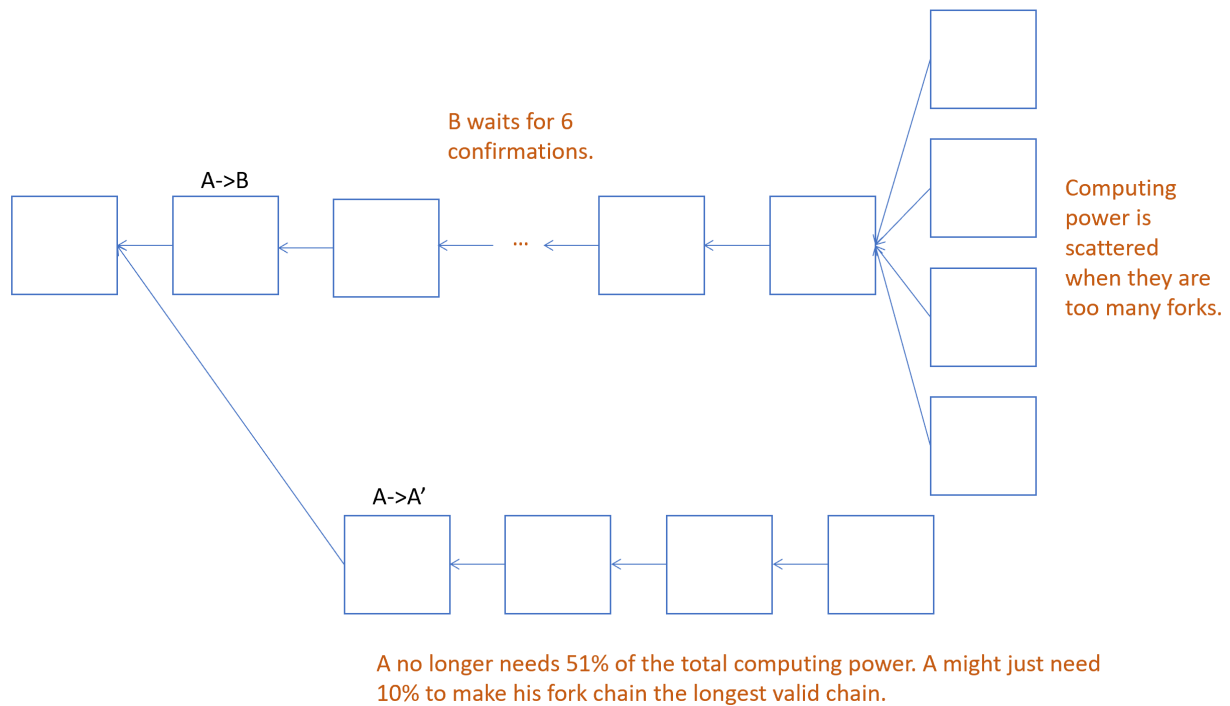


Fig 34

Therefore, when it comes to block time, shorter is not necessarily better. The key is to maintain stability—that is, to maintain a stable block time. Does this mean that Bitcoin’s current 10-minute block time is optimal? No, not necessarily. If it were changed to eight minutes, it should still work.

In fact, many people believe that Bitcoin’s 10-minute block time is too long for an efficient transaction system because no one wants to wait at least 10 minutes after making a payment, not to mention waiting for another six confirmations.

Ethereum, which will be discussed later, is designed to have a block time of 15 seconds. Therefore, Ethereum designed a new consensus protocol called **GHOST**. Orphan blocks generated in the Ethereum network cannot simply be discarded; a certain amount of uncle reward must be given. Ethereum also needs to adjust the mining difficulty to keep the block generation time at approximately 15 seconds.

There is no strict rule governing how short the average block time should be, but it must remain stable and cannot be reduced indefinitely. So, how exactly is the mining difficulty adjusted? The Bitcoin protocol stipulates that the mining difficulty—that is, the target range—should be adjusted every 2,016 blocks, or approximately every two weeks.

$$\frac{2016 \times 10 \text{ mins}}{60 \text{ mins} \times 24 \text{ hrs}} = 14 \text{ days}$$

$$\text{target} = \text{target} \times \frac{\text{actual time (time spent mining the last 2016 blocks)}}{\text{expected time (2016} \times 10\text{min)}}$$

For example, if the actual time required to generate 2,016 blocks is three weeks, then three weeks divided by two weeks equals 1.5. The new target value will therefore be 1.5 times the old target value. As the target value increases, the mining difficulty decreases. Note that there is a limit to how much the target can increase or decrease. The maximum adjustment factor is four, meaning that the target can increase or decrease by no more than a factor of four.

Although the mining difficulty adjustment mechanism is written into the code, Bitcoin is open source. What if a malicious node deliberately refuses to adjust the difficulty? There is no need to worry too much because honest nodes will not accept blocks mined by a malicious node using an incorrect difficulty level.

Recall that the block header contains a field called nBits. This field is 4 bytes in size. The nBits field is a compressed representation of the target. In other words, the target is not stored directly in the block header because it is too large. The target is 256 bits long and requires 32 bytes. Therefore, the nBits field indicates the difficulty used to mine the block. Other nodes can determine that the malicious node has not adjusted the target correctly and reject its block.

In the Ethereum mining system discussed later, the difficulty may be adjusted after each new block, and the adjustment method is much more complicated than Bitcoin's. Before the Bitcoin system appeared, there were already many cryptocurrency designs, but they all failed. Compared with those earlier designs, the Bitcoin system may be more "inconvenient" and highly conservative with regard to block time and block size. The current Bitcoin design may not necessarily be optimal.

Figure 35 shows the adjustment curve of Bitcoin's mining difficulty. The curve rises in a stepwise manner, corresponding to the two-week adjustment periods. The difficulty chart also shows that more and more people are participating in mining and that its popularity is increasing. If the difficulty of a particular cryptocurrency continues to decrease, it generally means that the cryptocurrency is losing popularity and may soon die out.

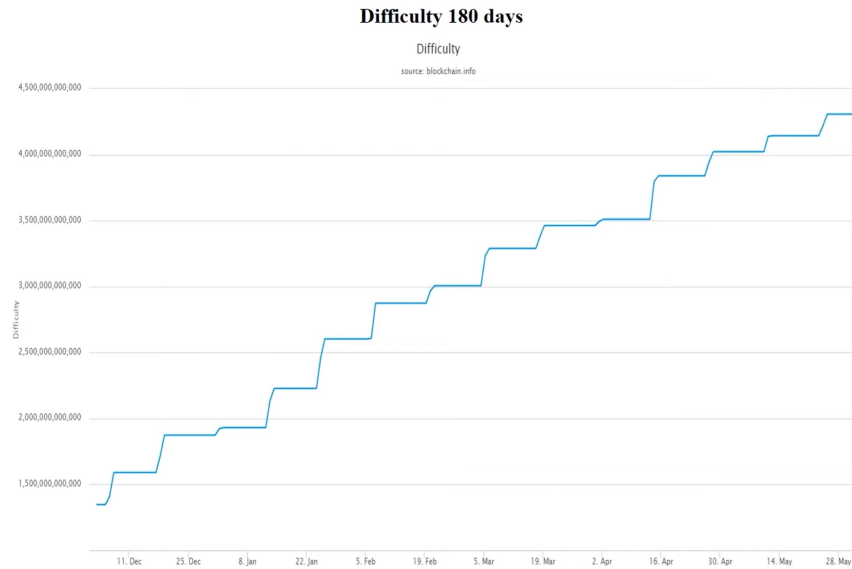


Fig 35

Figure 36 shows the block time, which generally fluctuates around 10 minutes.

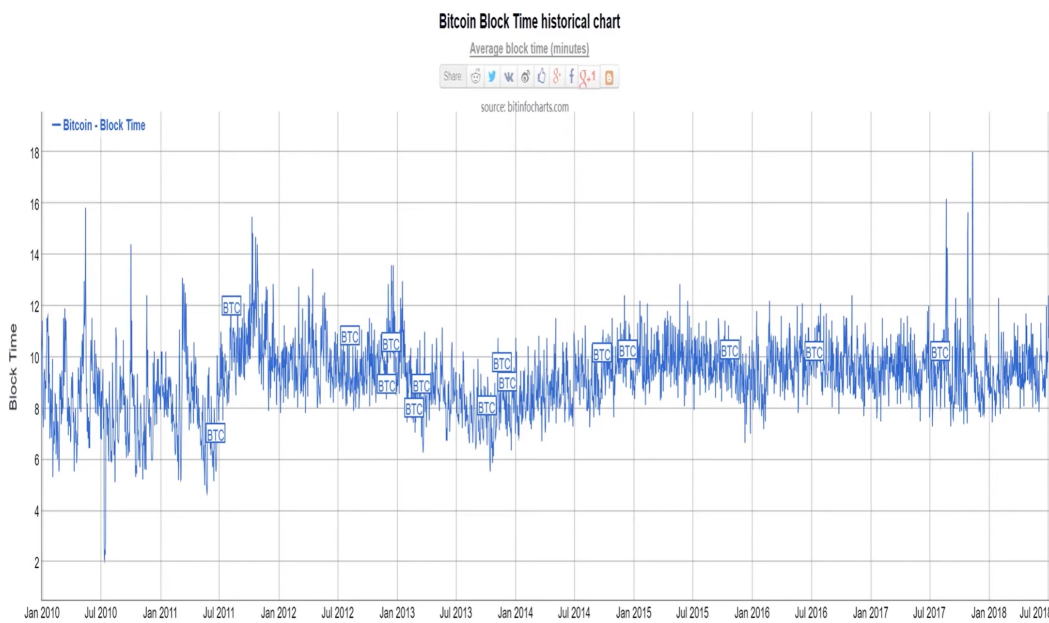


Fig 36

Miner & Mining pool

Mining hardware is becoming increasingly specialized. In the earliest days, ordinary PC CPUs could be used for mining. However, using them for this purpose was very wasteful, especially when buying a computer solely for mining. This is because only a small portion of the

computer's resources is used during mining, while most of its components remain idle. Calculating hash values requires only a small set of CPU instructions, and the hard drive also remains idle. Therefore, buying an entire computer solely for mining is not cost-effective.

Mining equipment consequently began to evolve from CPUs to GPUs. GPUs are primarily used for large-scale parallel computing. For example, deep learning requires powerful GPUs to perform large numbers of matrix multiplications. However, GPU mining is just as wasteful as CPU mining because many parts of the GPU, such as those used for floating-point calculations, remain idle during mining. These floating-point calculations are very useful for deep learning, but Bitcoin mining uses only integer calculations. GPUs have become more expensive partly because of Bitcoin mining. However, as the Bitcoin mining difficulty increases, GPU mining is no longer as cost-effective as it once was. The mining process is exceeding the computing capabilities of GPUs, and GPU mining is also very noisy.

Nowadays, **ASICs**, or application-specific integrated circuits, are commonly used for mining. These chips are designed specifically for mining, with all unnecessary functions removed. Therefore, they are highly cost-effective. Note that if an ASIC is designed for a particular cryptocurrency, it can mine only that cryptocurrency unless other cryptocurrencies use the same mining puzzle.

Some newly issued cryptocurrencies deliberately adopt an already popular mining puzzle, such as the same one used by Bitcoin. In this way, more people can easily mine these new coins. This situation is called **merged mining**. Except in cases of merged mining, each ASIC can serve only a specific cryptocurrency. ASIC chips have a long development cycle. For example, developing a Bitcoin ASIC may take at least one year, although this development speed is already very fast compared with that of other general-purpose chips.

A sharp drop in the price of Bitcoin can cause miners to lose money. However, when Bitcoin's price rises rapidly, mining competition becomes more intense. Miners may have to upgrade their ASIC machines every few months. Research shows that most of the profits generated by new ASIC mining machine models are earned within two months of their release because newly released machines have the strongest hash rates at that time.

Therefore, the timing of ASIC mining machine purchases is very important. Generally, ASIC mining machines are preordered. If the manufacturer cannot deliver the machines on time, miners can suffer serious losses. Some dishonest mining machine manufacturers use a new generation of machines to mine for two months before delivering them. They may also claim that they are testing the machines for customers before delivering them to the miners.

This improper behavior can be detected by observing changes in Bitcoin's total computing power. If the computing power suddenly increases, it may mean that a new generation of ASIC mining machines has been released. In fact, mining machine manufacturers may be more profitable than ordinary miners.

Some people believe that this arms race involving ASIC chips promotes centralization and goes against Bitcoin's original purpose. Therefore, some cryptocurrencies have adopted alternative, ASIC-resistant mining puzzles to counter the growing use of ASICs. The purpose of this design is to allow ordinary computers to participate in mining.

Another trend in mining is the emergence of mining pools. Individual miners cannot be guaranteed stable returns, even if they use the latest ASIC mining machines. An individual miner may mine only one block per year. As a result, individual mining is somewhat similar to buying a lottery ticket, and the income is unstable. In addition, as full nodes, these miners must perform various other duties.

Therefore, mining pools have emerged. The idea is simple: a mining pool organizes individual miners into a group. A mining pool generally consists of a full node that coordinates many mining machines. Each miner operates under the supervision of the mining pool manager. The miners are responsible only for calculating hash values, while the manager handles the other responsibilities of the full node, such as monitoring transactions, assembling and organizing candidate blocks, and monitoring the publication of blocks on the Bitcoin network. ASIC chips can calculate only hash values and cannot perform the other functions required of a full node.

The emergence of mining pools helps individual miners earn a stable income, but how should these miners divide the block reward? If the miners belong to the same company or individual, that company or person receives all the profits. But what if the miners in a mining pool are located around the world?

In this case, the miners and the manager must use the mining pool's communication protocol to distribute the task of calculating hashes. After a miner completes a calculation, the result is returned to the manager. When the manager receives the reward, everyone receives a portion of it.

How exactly is the reward distributed? We need a way to measure the work performed by each miner and allocate the block reward according to the amount of work completed. The income of individual miners is unstable because mining is extremely difficult. Therefore, we can reduce the difficulty within the mining pool. For example, if a valid hash value must begin with 70 zeros, we can reduce the pool's requirement to 60 zeros. A result that satisfies this easier requirement is called a **share**. This share is also called an almost-valid block.

Miners submit these almost-valid blocks to the manager whenever they find them. Note that an almost-valid block is used only by the manager to calculate the amount of work performed by each miner; it has no other use. In other words, after the mining pool mines a block, the block reward is distributed proportionally according to the number of almost-valid blocks submitted by each miner.

Now, imagine that a miner finds the next block. Is it possible for the miner to intentionally withhold the block and publish it independently to claim the block reward? The answer is no.

The miner's task is assigned by the manager, who is responsible for assembling the candidate block and then giving it to the miners so that they can try different nonce values.

As mentioned earlier, adjusting the nonce field may not be sufficient, so the coinbase field must also be adjusted. Therefore, the manager may give different coinbase values to different miners. The coinbase transaction in the candidate block provided by the manager contains a recipient address that belongs to the manager. The miner cannot steal the block reward because the recipient address is the manager's address.

Now, imagine another possibility. What if the miner ignores the candidate block initially provided by the manager, privately assembles another block, and changes the recipient address to their own? This will not work either. When the miner submits a share to the manager, the manager will reject it because the value of the coinbase transaction field has changed.

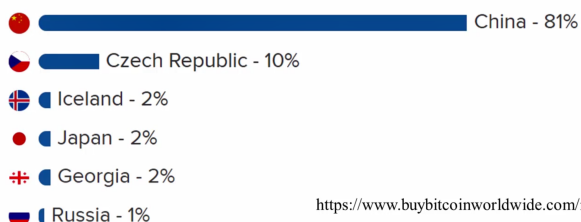
Can miners deliberately cause trouble for the manager? For example, a miner may find the next block but discard it instead of submitting it. This is possible, although it harms everyone in the pool. However, because mining pools compete with one another, spy miners may use this method to attack rival mining pools. These spy miners usually submit shares but discard valid blocks. It is like getting paid without working.

Figure 37 shows mining power statistics from 2018. In 2021, China lost its leading position in the competition for the right to append blocks. The United States now ranks first in the world.

Pool Concentration in China

Before we get into the best mining pools to join, it's important to note that [most mining pools are in China](#). Many only have Chinese websites and support. Mining centralization in China is one of Bitcoin's biggest issues at the moment.

There are about 20 major mining pools. Broken down by the percent of hash power controlled by a pool, and the location of that pool's company, we [estimate](#) that Chinese pools control ~81% of the network hash rate:



<https://www.buybitcoinworldwide.com/mining/pools/>

Fig 37

In 2014, a mining pool called GHash controlled more than 51% of the total computing power, which gave it the ability to launch a 51% attack (Figure 38). The mining pool voluntarily split itself up to support the long-term development of Bitcoin.

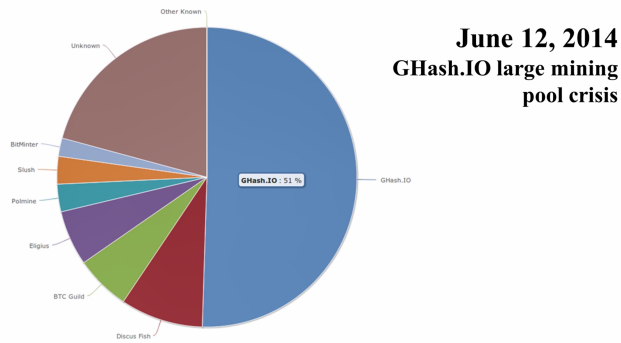


Fig 38

Figure 39 shows the distribution of mining pools in 2018. We can see that each mining pool accounts for only a small proportion of the total computing power, but this distribution may be superficial. Suppose an organization controls more than half of the total computing power. It does not have to place all its computing power in a single mining pool. Instead, it can distribute the computing power among several mining pools, thereby concealing the fact that it controls 51% of the total.

When necessary, the organization can combine its computing power distributed across different pools and launch an attack. Miners can easily join a mining pool by following its protocol. The pool manager sends assembled blocks to the miners, and the miners try different nonce values. Miners can also easily switch between mining pools.

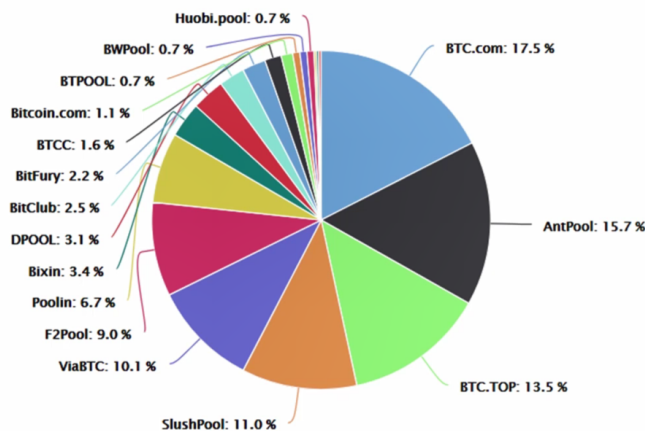


Fig 39

Without mining pools, an entity that wants to launch a 51% attack must pay enormous hardware costs to purchase enough mining machines to control more than half of the total

computing power. However, with the emergence of mining pools, the entity does not need to pay such high hardware costs. It only needs to attract enough miners to join its mining pool before attacking the blockchain.

A mining pool owner generally charges miners a management fee. However, a malicious pool may waive the management fee or even pay miners to join. Therefore, the emergence of large mining pools has created certain security concerns for the blockchain.

If a mining pool attracts more than 51% of the total computing power, what types of attacks can it launch? One possibility is a forking attack. For example, suppose A transfers 10,000 bitcoins to B. B waits for six confirmations and believes that the transaction is secure. At this point, A launches a forking attack and transfers the bitcoins to himself (Figure 40). Because A controls more than 51% of the computing power, A's chain will grow faster than the original chain. A can therefore make his chain the longest valid chain and reverse the transaction from A to B.

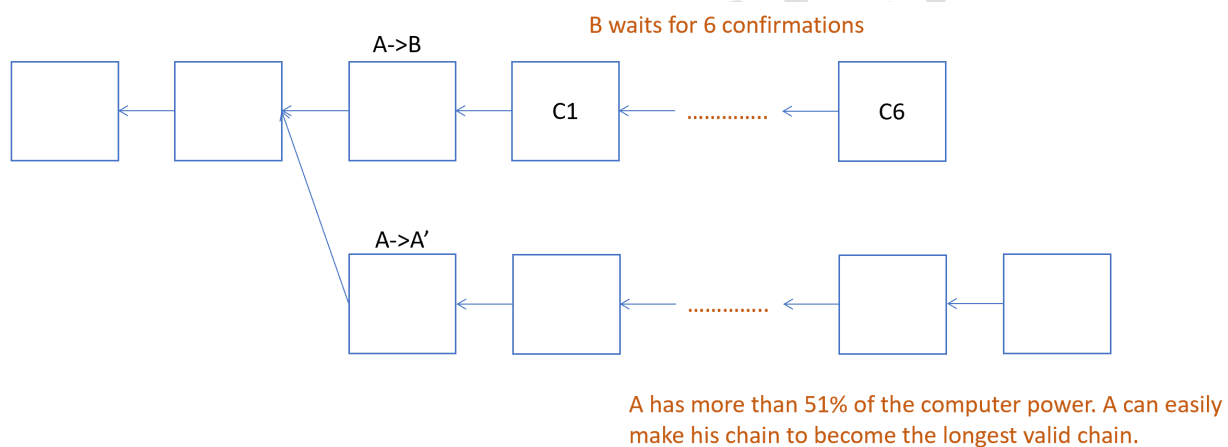


Fig 40

Note that 51% is not a strict threshold. This does not mean that an attack can be launched only after someone controls 51% of the computing power. It simply means that after reaching 51%, the probability of launching a successful attack becomes much greater.

In addition, the mining pool can engage in another malicious activity called a boycott. For example, suppose the mining pool wants to boycott A. Someone publishes a block containing the transaction from A to B on the blockchain. The mining pool can then prepare a block that does not contain the transaction from A to B and launch a boycott forking attack.

As mentioned earlier, when there are many honest nodes and only a few malicious nodes, there is still a possibility that the malicious nodes will obtain the right to append a block.

Malicious nodes can deliberately exclude certain transactions, but other honest nodes can include those transactions in future blocks.

However, if the malicious nodes control more than 51% of the computing power, they have a greater chance of obtaining the right to append the next block. Whenever a transaction involving A appears, the malicious nodes will immediately create a fork. In this way, other honest miners will not dare to include A's transactions because, if they do, their hard-earned blocks will be attacked through forking.

Can a mining pool with more than 51% of the computing power steal someone else's bitcoins? It cannot do so because it does not know the other person's private key. What if the mining pool forcibly includes transactions without valid signatures on the blockchain? This will create a fork, and the honest nodes will recognize the original chain and continue mining on it. Therefore, the mining pool cannot steal the bitcoins.

Bitcoin Network

Let's first discuss some basic concepts related to networks.

A network is a structure in which two or more devices, such as computers, printers, and routers, communicate with one another through communication media, such as network cables, optical fibers, and radio waves.

The Internet is a network of networks. Different networks are connected by routers. The round disks in Figure 41 below represent routers.

Internet communication is generally divided into two models: the client-server model and the peer-to-peer (P2P) model.

Let's first look at the client-server model. A client is the party that requests a service. For example, when you use your computer to search the Internet, your computer is the client because it requests a service. When you use a browser to search for information, the browser is the client because it requests the service. Therefore, a client can be either your computer or a program running on it.

A server is the party that provides services in response to requests sent by your computer. For example, if you want to open the Google website, a Google server will respond to your request. The client-server model is a typical centralized structure because Google can unilaterally decide to shut down its servers and prevent users from accessing the service.

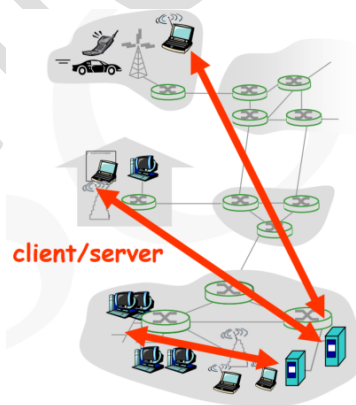


Fig 41

In the P2P model, all computers are equal, meaning that each computer acts as both a client and a server (Figure 42). They all provide services to one another. BitTorrent downloading is an example of a P2P network.

This model has a decentralized structure because shutting down one computer will not cause a problem, as other computers can continue providing the service. The Bitcoin system uses the P2P model.

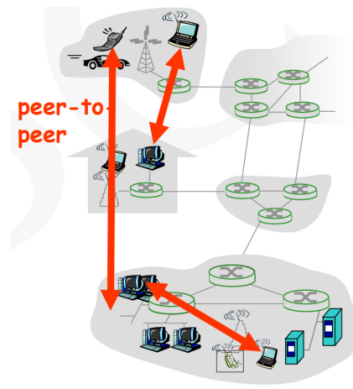


Fig 42

P2P Network

The Bitcoin program operates at the application layer, and its underlying layer is a P2P overlay network (Figure 43). Users send transactions to the Bitcoin network. After receiving the transactions, nodes package them into blocks and then publish the blocks to the Bitcoin network. So, how do these newly published transactions and blocks propagate throughout the Bitcoin network?

Application layer: Bitcoin blockchain

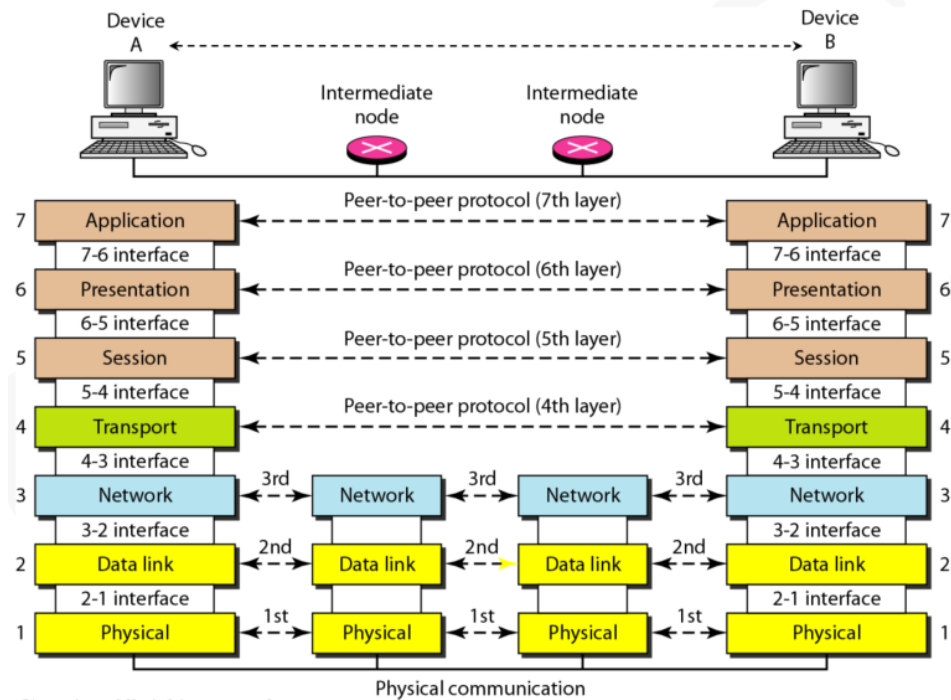
Network layer: P2P Overlay Network

Fig 43

Network Layering: You may have heard many times that a network is divided into layers. This does not mean that the network is physically layered. Communication between computers is not as simple as communication between people. Computers must follow certain protocols when communicating with one another.

To separate the functions of different protocols and maintain scalability, we divide a network into seven layers: the application layer, presentation layer, session layer, transport layer, network layer, data link layer, and physical layer. Each layer has its corresponding protocols (Figure 44).

Therefore, we say that Bitcoin is an application that runs at the application layer, while its underlying network uses the P2P model.



What exactly is a protocol? It is very simple. When people write letters to one another, they must follow a set of procedures to deliver them (Figure 45). This set of procedures is called a protocol.

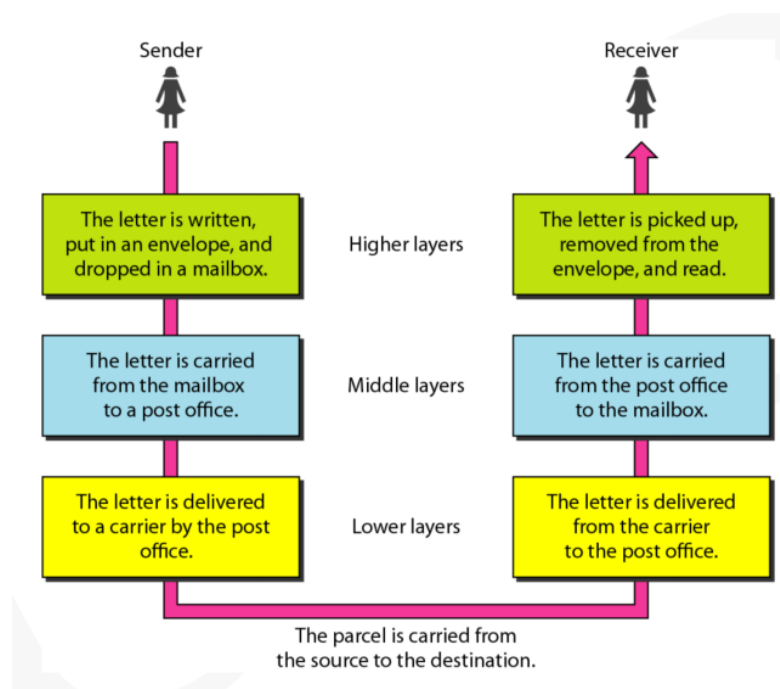


Fig 45

Here is another example in which people ask each other what time it is. The steps followed by both parties form the protocol (Figure 46).

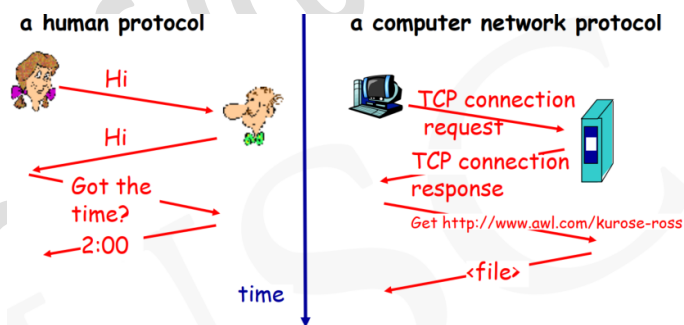


Fig 46

Bitcoin's P2P network is very simple, and all nodes are equal. Unlike Bitcoin, some other P2P networks have super nodes or master nodes. If you want to join the Bitcoin network, you only need to know a seed node. This seed node can tell you about the other nodes in the network that it knows. TCP communication between nodes is helpful for passing through firewalls.

You do not need to perform any special operation when leaving the Bitcoin network. You can simply exit the application, and the other nodes will delete your information if they do not receive any messages from you within a certain period.

TCP/IP: I believe you have encountered the term TCP/IP countless times. TCP and IP are among the protocols mentioned above. We need TCP/IP because we have connected various small “networks” to form the Internet, and these smaller networks may use different technical specifications. Without a common protocol, the different networks cannot easily communicate with one another.

You can think of these smaller networks as different countries whose people speak different languages. Clearly, they need a common language to communicate with one another. This common language is TCP/IP. All the smaller networks that form the Internet agree to follow the same TCP/IP communication rules.

Internet is a network of networks, each with different technologies, then how can H1 and H2 communicate when they are not in the same network?

Networks may have different technologies, but they all agree on TCP/IP; therefore the whole network is transparent to the outside world

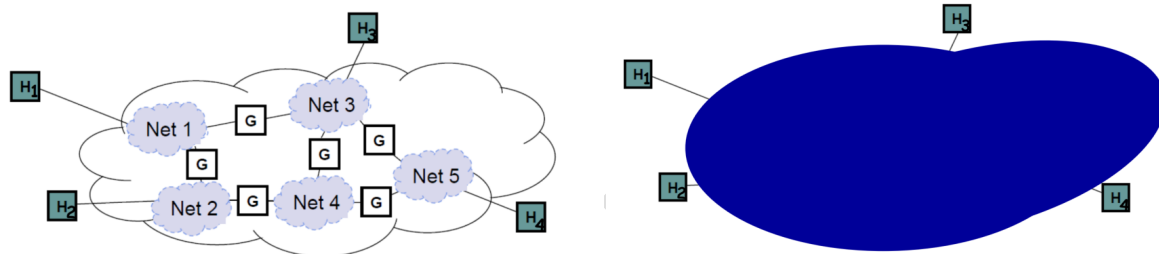


Fig 47

Under the TCP/IP model, the seven-layer network model mentioned above is simplified into five layers: the application layer, transport layer, network layer, data link layer, and physical layer. This five-layer model is called TCP/IP because the TCP protocol is mainly used at the transport layer, while the IP protocol is mainly used at the network layer. TCP and IP are both names of protocols. However, this does not mean that TCP and IP are the only protocols available at the transport and network layers; other protocols are also available.

The design principle of the Bitcoin network is to be simple and robust rather than efficient. Each node maintains a set of neighboring nodes. Messages are disseminated through flooding. When a node receives a message for the first time, it forwards the message to all its neighboring nodes. At the same time, the node records that it has received the message. If the node receives the same message again, it will not forward it to its neighboring nodes.

A node's neighboring nodes are selected randomly, and the underlying topology does not matter. If you are operating a node in Beijing, one of your neighboring nodes may be in New York. Therefore, transferring bitcoins to someone living downstairs from you takes approximately the same amount of time as transferring them to someone in Australia. This design improves robustness, but it does so at the expense of efficiency.

Flooding is a term used in computer networks. It means that when a node receives a message, it forwards that message to all the nodes it knows. The message spreads to all the neighboring nodes like a flood, so this process is called flooding.

Topology, a term used in computer networks, refers to the physical layout of a network. Some examples are shown below (Figure 48).

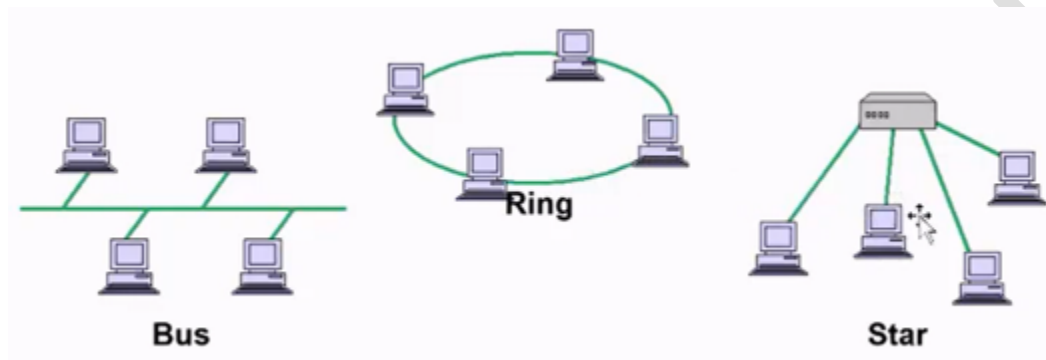


Fig 48

Each node also maintains a collection of transactions waiting to be written to the blockchain. When a node receives information about a transaction for the first time, it adds the transaction to its pending transaction collection and forwards the message to its neighboring nodes. Before forwarding the message, the node must ensure that the transaction is valid, has a valid signature, and has a valid source.

Note that we need to consider **race conditions**. For example, two conflicting transactions, both of which attempt to spend bitcoins from the same source, may be broadcast to the network at the same time. Depending on their locations in the network, some nodes may receive the transaction from A to B first, while others may receive the transaction from A to C first.

A race condition is a situation that may occur during the execution of a program. It specifically refers to a situation in which multiple programs attempt to operate on the same information, causing the final result to be uncertain. You can compare this situation to people rushing to board a bus without forming a line when it arrives. As a result, it is uncertain who will be able to board the bus first.

For example, if a node receives the transaction from A to B first, it adds that transaction to its pending transaction collection. If the node later receives the transaction from A to C, it considers the second transaction invalid and ignores it. However, if the transaction from A to C is received first, the transaction from A to B will be ignored.

Note that if transactions in the pending transaction collection have been written to the blockchain by other nodes, they must be deleted from the collection. For example, if node X learns that a newly published block contains the transaction from A to B, it deletes that transaction from its pending transaction collection.

What happens if nodes that have the transaction from A to B in their pending transaction collections discover that a newly published block contains the transaction from A to C? These nodes must delete the transaction from A to B. Because the transaction from A to C has been accepted in the newly published block, the transaction from A to B is now invalid and cannot remain in their pending transaction collections. This may happen if another node receives the transaction from A to C first, mines the next block, and publishes it.

The propagation of newly published blocks through the network is similar to the propagation of new transaction information. After receiving a newly published block, each node must check whether the contents of the block are valid and whether the block is appended to the longest valid chain. The larger the block, the slower its propagation speed.

The Bitcoin protocol stipulates that the maximum block size is 1 MB. This may seem very small, but it is already quite large for the Bitcoin network because the network's design consumes a considerable amount of bandwidth. Bandwidth is the bottleneck. A 1 MB block may take tens of seconds to propagate throughout the entire network.

A bit is a binary digit. It is the smallest unit of information storage in a computer. A bit represents one of two logical values: 1 or 0, true or false, and on or off. In other words, it represents two possible states. Bitcoin's name may be based on this concept. In computer circuits, we can distinguish between 1 and 0, true and false, or on and off by controlling voltage levels.

1 Byte = 8 bits

1 KB = 1024 Byte

1 MB = 1024 KB

Note that information is propagated through the Bitcoin network on a best-effort basis. In other words, when a transaction is published on the network, not every node will necessarily receive it. It is also possible for some nodes to forward invalid transactions.

Full Node & Light Node

Full node	Light node
• Always online • Maintains the entire blockchain on its local disk • Maintains the UTXO set in memory for quick validation	• Not always online • Maintains block headers only • Maintains only relevant transactions • Can validate only relevant transaction

• Listens for transactions on the Bitcoin network and validates each transaction • Decides which transactions will be included in a block • Listens for newly published blocks and validates them • Mines the next block	• Cannot validate a new block, but can validate its mining difficulty • Can identify only the longest chain, not the longest valid chain
---	---

A **full node** performs verification in three areas. First, it verifies the validity of each transaction, including coinbase transactions. Second, it hashes the block header to verify that it meets the difficulty requirements, such as having enough zeros at the beginning. It also checks whether the difficulty target value is correct and adjusts the mining difficulty every two weeks according to the protocol's requirements. Third, it verifies whether the block extends the longest valid chain.

Light nodes can verify the mining difficulty because they maintain block header information. A light node assumes that the full node that publishes a block will not include invalid transactions because there is usually no benefit to publishing them. A light node knows which chain is the longest, but it does not know which chain is the longest valid chain. Most nodes in the Bitcoin network are light nodes. If you only need to transfer bitcoins and do not need to mine, running a light node is sufficient.

When a node discovers while mining that someone else has mined a "new block," what should it do? The node should stop mining and reassemble its candidate block locally. If it wants to mine on top of the new block, the transaction contents must change. Some transactions in its candidate block may already have been included in the new block. The block header must also change because it contains the root hash value of the Merkle tree formed by the transactions, as well as a pointer to the previous block. The node then restarts mining on top of the new block.

It may seem that the node has performed a great deal of work without receiving anything in return. However, recall that Bitcoin mining is memoryless, or progress-free. Therefore, whether the node continues mining the original block or begins mining on top of someone else's new block, its probability of mining the next block remains the same. The amount of time it has spent mining in the past does not affect its probability of mining a block in the future.

When a node mines a block and publishes the block it has assembled, there is no guarantee that its block will win. Sometimes a race condition occurs. The node may also have included an invalid transaction, causing the other nodes to reject its block.

The security of the Bitcoin system is provided by two components: cryptography and the consensus mechanism. Cryptography ensures that other people cannot forge your signature or maliciously steal your bitcoins because they do not know your private key. Note that this

protection assumes that most nodes in the system are honest and will not accept transactions with invalid signatures.

xuanf@usc.edu

Bitcoin Script

Script Language

Bitcoin's scripting language is very simple and can access only a stack-based memory space. Unlike Java and C, it does not provide a comprehensive set of features. Therefore, Bitcoin's scripting language is called a stack-based language.

A programming language is a language that programmers use to write computer programs that machines can understand and execute. Programming languages, such as C, C++, Java, and Python, have their own grammatical rules. They are similar to human languages, such as English. English follows certain grammatical rules, and you must follow these rules so that other English speakers can understand you. Programming languages also follow grammatical rules and allow humans to communicate instructions to machines. A so-called scripting language is simply a highly simplified programming language.

A script is just a piece of code. Bitcoin's scripting language is Turing-incomplete and does not have loop structures. Its instructions can only be executed sequentially, although branches are allowed. The language is based on a stack.

A stack is a basic data structure in computer science (Figure 49). Its main characteristic is that the first item placed into it is the last one that can be removed. Imagine that you have a deep box. You place plate number 1 into the box, followed by plate number 2, plate number 3, and so on. If you later want to remove plate number 1, you must first remove plate number 5, followed by plate number 4, and so on.

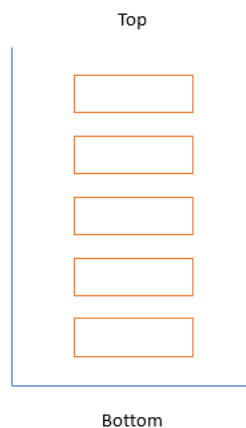


Fig49

Some script commands are shown below (Figures 50 and 51).

Flow control

Word	Opcode	Hex	Input	Output	Description
OP_NOP	97	0x61	Nothing	Nothing	Does nothing.
OP_IF	99	0x63	<expression> if [statements] [else [statements]]* endif		If the top stack value is not False, the statements are executed. The top stack value is removed.
OP_NOTIF	100	0x64	<expression> notif [statements] [else [statements]]* endif		If the top stack value is False, the statements are executed. The top stack value is removed.
OP_ELSE	103	0x67	<expression> if [statements] [else [statements]]* endif		If the preceding OP_IF or OP_NOTIF or OP_ELSE was not executed then these statements are and if the preceding OP_IF or OP_NOTIF or OP_ELSE was executed then these statements are not.
OP_ENDIF	104	0x68	<expression> if [statements] [else [statements]]* endif		Ends an if/else block. All blocks must end, or the transaction is invalid . An OP_ENDIF without OP_IF earlier is also invalid .
OP_VERIFY	105	0x69	True / false	Nothing / fail	Marks transaction as invalid if top stack value is not true. The top stack value is removed.
OP_RETURN	106	0x6a	Nothing	fail	Marks transaction as invalid . Since bitcoin 0.9, a standard way of attaching extra data to transactions is to add a zero-value output with a scriptPubKey consisting of OP_RETURN followed by data. Such outputs are provably unspendable and specially discarded from storage in the UTXO set, reducing their cost to the network. Since 0.12.0, standard relay rules allow a single output with OP_RETURN, that contains any sequence of push statements (or OP_RESERVED ⁽¹⁾) after the OP_RETURN provided the total scriptPubKey length is at most 83 bytes.

Fig 50

Stack

Word	Opcode	Hex	Input	Output	Description
OP_TOALTSTACK	107	0x6b	x1	(alt)x1	Puts the input onto the top of the alt stack. Removes it from the main stack.
OP_FROMALTSTACK	108	0x6c	(alt)x1	x1	Puts the input onto the top of the main stack. Removes it from the alt stack.
OP_IFDUP	115	0x73	x	x / x x	If the top stack value is not 0, duplicate it.
OP_DEPTH	116	0x74	Nothing	<Stack size>	Puts the number of stack items onto the stack.
OP_DROP	117	0x75	x	Nothing	Removes the top stack item.
OP_DUP	118	0x76	x	x x	Duplicates the top stack item.
OP_NIP	119	0x77	x1 x2	x2	Removes the second-to-top stack item.
OP_OVER	120	0x78	x1 x2	x1 x2 x1	Copies the second-to-top stack item to the top.
OP_PICK	121	0x79	xn ... x2 x1 x0 <n>	xn ... x2 x1 x0 xn	The item <i>n</i> back in the stack is copied to the top.
OP_ROLL	122	0x7a	xn ... x2 x1 x0 <n>	... x2 x1 x0 xn	The item <i>n</i> back in the stack is moved to the top.
OP_ROT	123	0x7b	x1 x2 x3	x2 x3 x1	The 3rd item down the stack is moved to the top.
OP_SWAP	124	0x7c	x1 x2	x2 x1	The top two items on the stack are swapped.
OP_TUCK	125	0x7d	x1 x2	x2 x1 x2	The item at the top of the stack is copied and inserted before the second-to-top item.
OP_2DROP	109	0x6d	x1 x2	Nothing	Removes the top two stack items.
OP_2DUP	110	0x6e	x1 x2	x1 x2 x1 x2	Duplicates the top two stack items.
OP_3DUP	111	0x6f	x1 x2 x3	x1 x2 x3 x1 x2 x3	Duplicates the top three stack items.
OP_2OVER	112	0x70	x1 x2 x3 x4	x1 x2 x3 x4 x1 x2	Copies the pair of items two spaces back in the stack to the front.
OP_2ROT	113	0x71	x1 x2 x3 x4 x5 x6	x3 x4 x5 x6 x1 x2	The fifth and sixth items back are moved to the top of the stack.
OP_2SWAP	114	0x72	x1 x2 x3 x4	x3 x4 x1 x2	Swaps the top two pairs of items.

Fig 51

Let's look at a simple script example (Figure 52). We begin with an empty stack. First, we push 10 onto the stack and then push 3 onto it. Next, we execute SUB, the subtraction instruction. This instruction removes the top two elements from the stack, performs the subtraction, and pushes the result back onto the stack.

We then push another 7 onto the stack and execute EQUAL. This instruction removes the top two numbers and compares them. In this case, $(7=7)$, so the two numbers are equal. The instruction returns 1, which is then pushed onto the stack.

Finally, we execute VERIFY. This instruction examines the value at the top of the stack. If the value is 1, the verification succeeds.

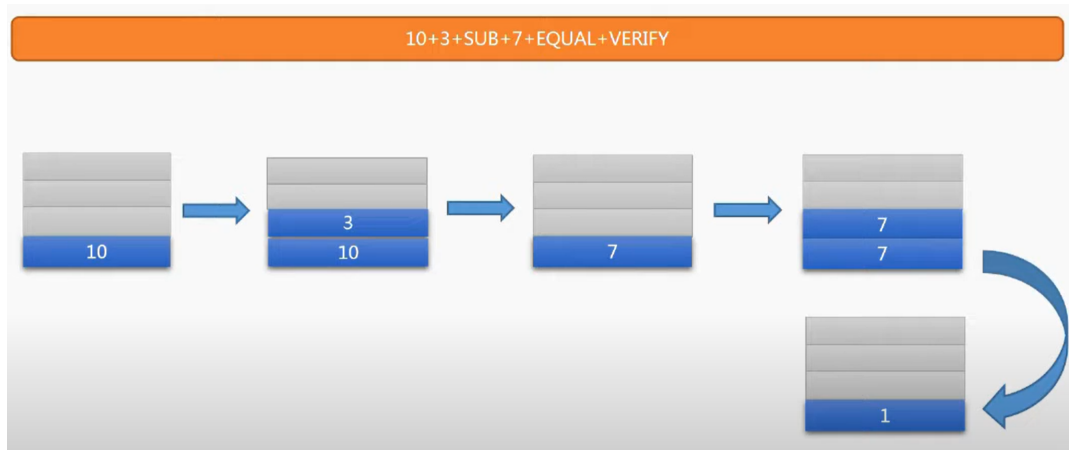


Fig 52

Let's look at a real Bitcoin script example (Figure 53). Just as we used the simple example above to verify whether 7 is equal to 7, we use this script to verify the validity of the public key and signature.

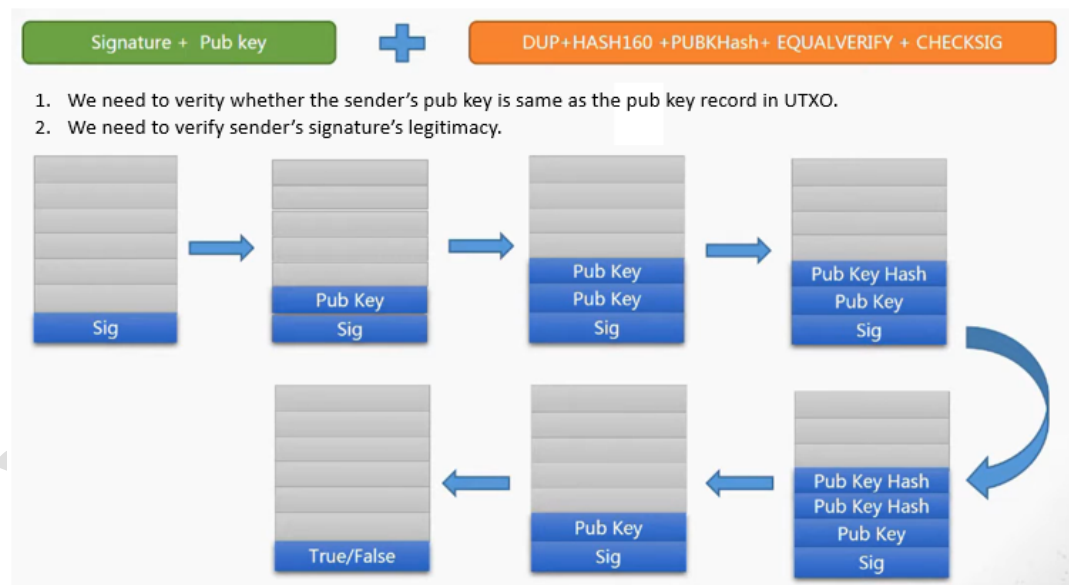


Fig 53

Now, let's look at a transaction example (Figure 54).

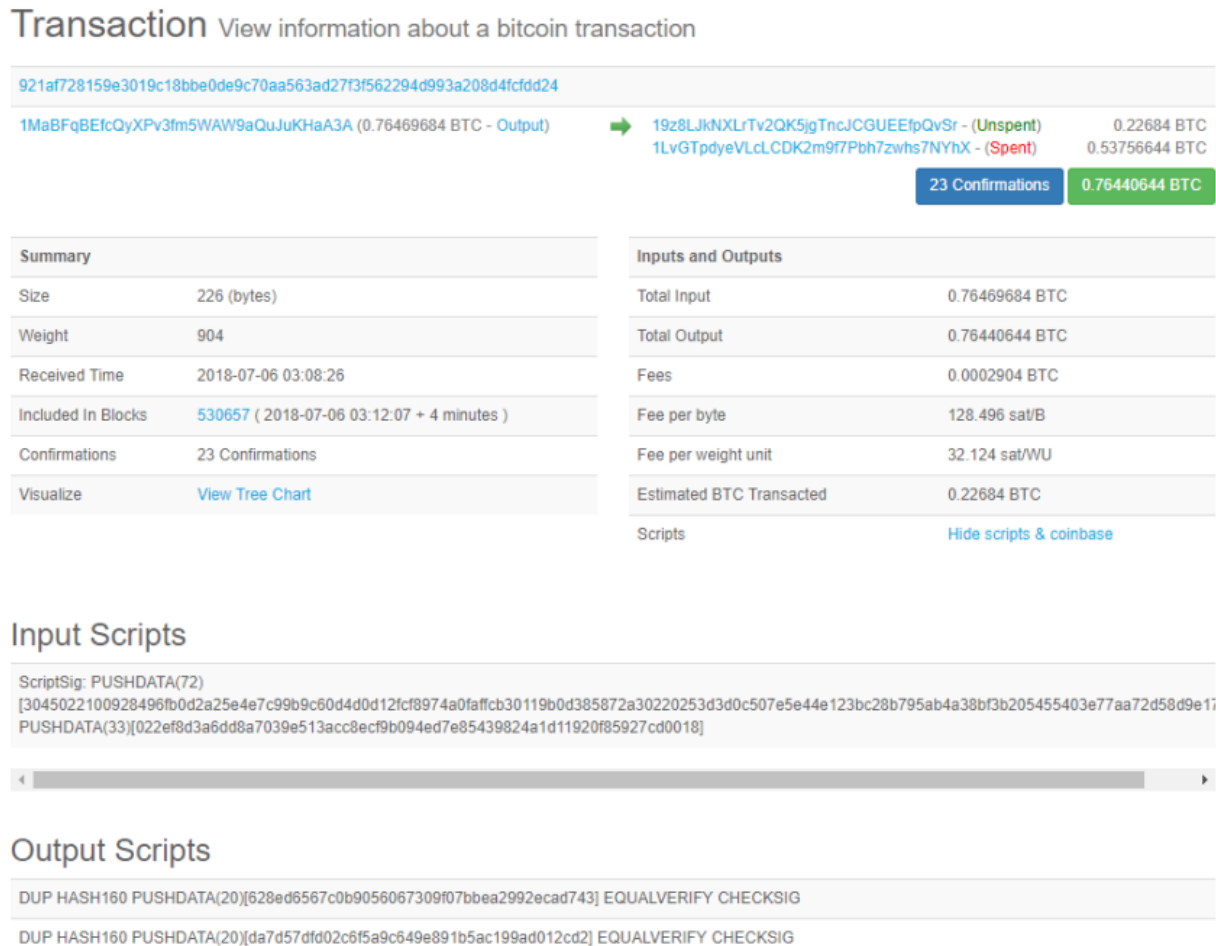


Fig 54

The transaction has one input and two outputs. Although “output” is written on the left, it is actually the input of this transaction. The word “output” is used here because the bitcoins used in this transaction are the output of a previous transaction.

Of the two outputs on the right, the upper one is unspent, while the lower one has been spent. The transaction has 23 confirmations, so the probability of it being reversed is very low.

The input script consists of two operations, each of which pushes two very long numbers onto the stack. The output section contains two lines corresponding to the two outputs shown above because each output has its own separate script.

Let's look at the structure of this transaction (Figure 55).

```
"result": {
  "txid": "921a...dd24",
  "hash": "921a...dd24",
  "version": 1,
  "size": 226,
  "locktime": 0,
  "vin": [...],
  "vout": [...],
  "blockhash": "0000000000000000000000000000000000000000000000000000000000000000",
  "confirmations": 23,
  "time": 1530846727,
  "blocktime": 1530846727
}
```

Fig 55

“txid” is the transaction ID.

“hash” is the hash value of the transaction.

“version” is the version number of the Bitcoin protocol.

“size” is the size of the transaction.

“locktime” refers to the time at which the transaction becomes effective. A value of 0 means that it is effective immediately.

“vin” and “vout” are the inputs and outputs, respectively.

“blockhash” is the hash value of the block containing the transaction.

“confirmations” is the number of confirmations the transaction has received.

“time” is the time at which the transaction was generated, expressed in seconds.

“blocktime” is the time at which the block containing the transaction was generated, expressed in seconds.

The following shows the input of the transaction (Figure 56).

```

    "vin": [{
      "txid": "c0cb...c57b",
      "vout": 0,
      "scriptSig": {
        "asm": "3045...0018",
        "hex": "4830...0018"
      },
    },
  ]],

```

Fig 56

A transaction can have multiple inputs, but our example has only one. For each input, it is necessary to identify its source and provide a signature. The txid in the figure above represents the hash value of the “previous transaction,” while vout indicates the number of the output in that previous transaction.

scriptSig is the input script. The simplest form of an input script is just a signature proving that you have the right to spend the bitcoins. This means that a Bitcoin transaction may require multiple signatures.

The following shows the outputs of the transaction (Figure 57).

```

"vout": [{
  "value": 0.22684000,
  "n": 0,
  "scriptPubKey": {
    "asm": "DUP HASH160 628e...d743 EQUALVERIFY CHECKSIG",
    "hex": "76a9...88ac",
    "reqSigs": 1,
    "type": "pubkeyhash",
    "addresses": [ "19z8LJkNXLrTv2QK5jgTncJCGUEEfpQvSr" ]
  }
}, {
  "value": 0.53756644,
  "n": 1,
  "scriptPubKey": {
    "asm": "DUP HASH160 da7d...2cd2 EQUALVERIFY CHECKSIG",
    "hex": "76a9...88ac",
    "reqSigs": 1,
    "type": "pubkeyhash",
    "addresses": [ "1LvGTpdyeVLcLCDK2m9f7Pbh7zwhs7NYhX" ]
  }
}],

```

Fig 57

There are two outputs.

value is the value of the output, indicating how many bitcoins are transferred to the other party.

n is the serial number.

scriptPubKey is the output script. The simplest form of an output script provides a public key.

asm is the content of the output script.

reqSigs indicates how many signatures are required for the output.

type is the output type.

pubkeyhash is the hash of the public key.

addresses is the output address.

Let's look at how the input and output scripts are executed (Figure 58). In the following blockchain, A sends bitcoins to B, and B then sends them to C. In other words, the bitcoins in the transaction from B to C come from the transaction from A to B.

Therefore, the txid and vout in the input of the transaction from B to C point to the corresponding output of the transaction from A to B. To verify the validity of the transaction from B to C, its input script must be combined with the output script of the transaction from A to B.

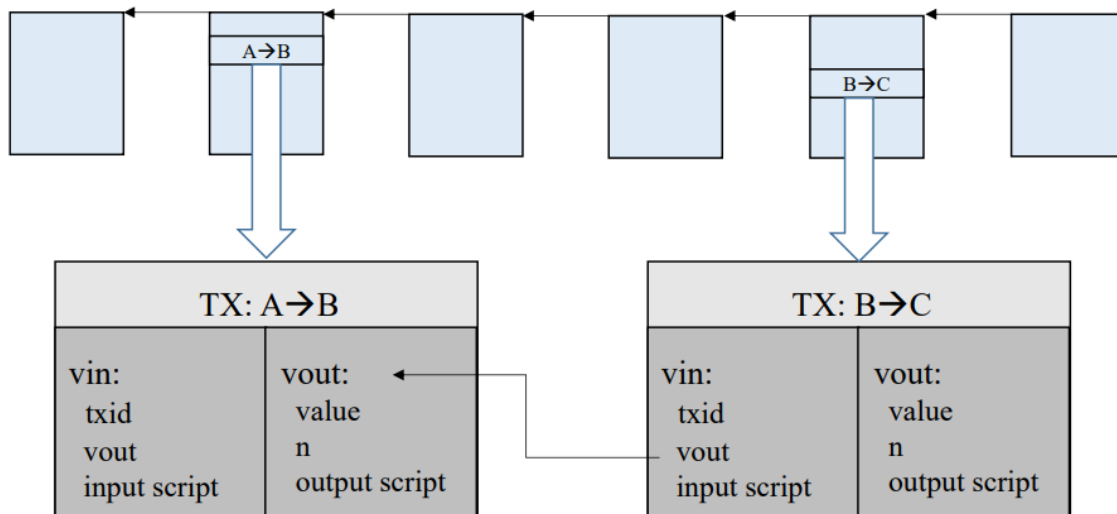


Fig 58

In early versions of Bitcoin, these two scripts were combined and executed from beginning to end (Figure 59). Later, because of certain security considerations, they began to be executed separately. The input script is executed first, and if no errors occur, the output script is then executed.

If everything executes successfully and the final value at the top of the stack is nonzero, the result is considered true. The verification therefore succeeds, and the transaction is valid. If any error occurs, the transaction is invalid.

If a transaction has multiple inputs, each input script must be verified together with its corresponding output script. The transaction is valid only if all the verifications succeed.

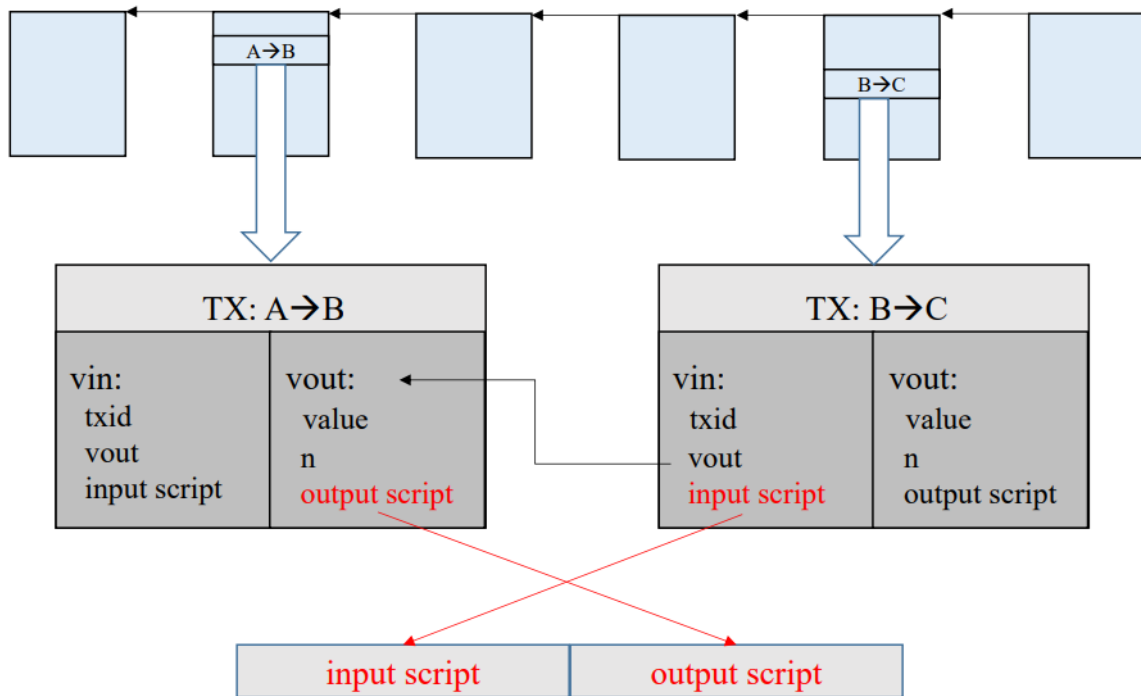


Fig 59

Script Types

The simplest form is called **Pay to Public Key** (Fig 60)

P2PK (Pay to Public Key)

input script:

```
PUSHDATA (Sig)
```

output script:

```
PUSHDATA (PubKey)
```

```
CHECKSIG
```

Fig 60

The receiver's public key is provided directly in the output script. CHECKSIG is the operation that verifies the signature. The signature is created by using the private key to sign the entire transaction. This form is the simplest because the public key is provided directly in the output script.

After the input and output scripts are combined, the resulting script looks as follows (Figure 61). The first line comes from the input script, while the last two lines come from the output script. Note that the scripts are actually executed separately; we have combined them here for convenience.

```
PUSHDATA (Sig)
```

```
PUSHDATA (PubKey)
```

```
CHECKSIG
```

Fig 61

The first line pushes the input signature onto the stack. The second line pushes the output public key onto the stack. The third line removes the two elements from the stack and uses the public key to verify whether the signature is correct. If the signature is correct, the transaction is valid.

Another type is called **Pay to Public Key Hash** (Fig 62).

P2PKH (Pay to Public Key Hash)

input script:

```
PUSHDATA (Sig)  
PUSHDATA (PubKey)
```

output script:

```
DUP  
HASH160  
PUSHDATA (PubKeyHash)  
EQUALVERIFY  
CHECKSIG
```

Fig 62

The receiver's public key is not provided directly in the output script; only the hash of the public key is provided. The public key is provided in the input script, which contains both the signature and the public key. DUP and HASH160 in the output script are used to verify the correctness of the signature. P2PKH is the most commonly used script type.

Once again, the combined input and output script looks as follows (Figure 63). The first two lines come from the input script, while the remaining lines come from the output script.

The first line pushes the signature onto the stack. The second line pushes the public key onto the stack. The third line copies the element at the top of the stack, leaving an additional copy of the public key there. HASH160 then removes the top element from the stack, hashes it, and pushes the resulting hash value onto the stack. Therefore, the hash value of the public key is now at the top of the stack.

```
PUSHDATA (Sig)  
PUSHDATA (PubKey)  
DUP  
HASH160  
PUSHDATA (PubKeyHash)  
EQUALVERIFY  
CHECKSIG
```

Fig 63

PUSHDATA pushes the hash value of the public key provided in the output script onto the stack. At this point, the stack contains two hash values. EQUALVERIFY removes these two elements and compares them to determine whether they are equal. The purpose of this step is to prevent someone from using their own public key to impersonate the receiver.

CHECKSIG removes two elements from the stack and uses the public key to verify whether the signature is correct. If it is correct, the transaction is valid. Any error during the process above will cause the transaction to be considered invalid.

The most complex script type is called Pay to Script Hash (Fig 64).

P2SH (Pay to Script Hash)

BIP 16

input script:

...

PUSHDATA(Sig)

...

PUSHDATA(serialized redeemScript)

output script:

HASH160

PUSHDATA(redeemScriptHash)

EQUAL

Fig 64

This output script does not provide the hash of the receiver's public key. Instead, it provides the hash of a script supplied by the receiver. This script is called a **redeem script**.

When the bitcoins are spent in the future, the input script must provide the specific contents of the redeem script. It must also provide the signature required to execute the redeem script correctly.

The verification process consists of two steps. First, it is necessary to verify whether the redeem script provided in the input script matches the hash value provided in the output script. If they do not match, the provided redeem script is incorrect. If they match, the contents of the redeem script must then be executed to determine whether the script runs correctly. If both verification steps succeed, the transaction is valid.

Now, let's look at how P2SH can be used to implement P2PK (Figure 65).

redeemScript:

```
PUSHDATA (PubKey)  
CHECKSIG
```

input script:

```
PUSHDATA (Sig)  
PUSHDATA (serialized redeemScript)
```

output script:

```
HASH160  
PUSHDATA (redeemScriptHash)  
EQUAL
```

Fig 65

The input script provides the signature and the serialized redeem script. The redeem script itself provides the public key and then uses CHECKSIG to verify the signature. The output script is used to verify whether the redeem script provided in the input script is correct.

Figure 66 shows the first verification step.

```
PUSHDATA (Sig)  
PUSHDATA (seriRS)  
HASH160  
PUSHDATA (RSH)  
EQUAL
```

Fig 66

Let's first combine the input and output scripts. The first two lines come from the input script, while the remaining lines come from the output script.

The first line pushes the signature from the input script onto the stack. The second line pushes the redeem script onto the stack. The third line obtains the hash value of the redeem script, represented by RSH. The fourth line pushes the hash value provided in the output script onto the stack.

At this point, the stack contains two RSH values. EQUAL then compares them to determine whether they are equal. If they are not equal, the verification fails. If they are equal, the two hash values are removed from the stack.

The first verification step is now complete. Next, we begin the second step (Figure 67).

```
PUSHDATA (PubKey)  
CHECKSIG
```

Fig 67

We must deserialize the redeem script provided in the input script and then execute it. The redeem script pushes PubKey onto the stack and uses CHECKSIG to verify the signature provided in the input script. If the verification succeeds, the entire P2SH verification process is complete.

Multi-Signature

Why do we use a complicated script type such as P2SH? It is indeed unnecessarily complicated for simple transactions, and P2SH was not available in the original version of Bitcoin. This feature was later added through a soft fork.

The main use of this script type is to support multi-signature transactions. An output in the Bitcoin system may require multiple signatures before its bitcoins can be spent. For example, a large company's Bitcoin account may require signatures from at least three out of five senior executives before any bitcoins can be withdrawn. This provides protection against the loss or exposure of one person's private key. This function is implemented using CHECKMULTISIG.

The implementation works as follows. The output script provides n public keys and specifies a threshold value m. As long as the input script provides any m valid signatures corresponding to the n public keys, the verification succeeds. In the example above, m=3 and n=5.

Figure 68 shows the input and output scripts of a multi-signature transaction.

input script:

x

PUSHDATA(Sig_1)

PUSHDATA(Sig_2)

...

PUSHDATA(Sig_M)

outputScript:

M

PUSHDATA(pubkey_1)

PUSHDATA(pubkey_2)

...

PUSHDATA(pubkey_N)

N

CHECKMULTISIG

fig 68

There is an x in the first line of the input script because the implementation of CHECKMULTISIG in the Bitcoin code contains a bug that cannot be modified. This bug causes one extra element to be removed from the stack. Because Bitcoin is a decentralized system, fixing bugs through software upgrades is very costly. If this bug were changed, a hard fork would be required.

Therefore, the solution is to push a useless element onto the stack in the input script. The x represents this useless element. Also, note that the relative order of n and m must correspond correctly.

Figure 69 shows an execution example of CHECKMULTISIG, where m=2 and n=3. FALSE in the first line is the useless element that must be pushed onto the stack. The second and third lines push the signatures onto the stack. The execution of the input script ends at this point.

Next, the output script is executed. First, the threshold value m=2 is pushed onto the stack. The three public keys are then pushed onto the stack, followed by n=3. Finally, CHECKMULTISIG is executed to determine whether the stack contains two of the three required signatures. If it does, the verification succeeds.

```
FALSE
PUSHDATA (Sig_1)
PUSHDATA (Sig_2)
2
PUSHDATA (pubkey_1)
PUSHDATA (pubkey_2)
PUSHDATA (pubkey_3)
3
CHECKMULTISIG
```

Fig 69

Note that the process above does not use P2SH. Instead, it uses Bitcoin's native CHECKMULTISIG function. However, this native function is somewhat inconvenient. For example, when shopping online, one e-commerce company may state that it uses multiple signatures and requires three out of five signatures. Another e-commerce company may require four out of six signatures.

Therefore, when generating transfer information, users must satisfy the signature requirements of different e-commerce companies and provide the values of m and n . In other words, all this complicated information must be included in the user's output script. This is inconvenient because the complexity is exposed to the user.

Therefore, we need to use P2SH to implement multi-signature transactions (Figure 70). This essentially transfers the complexity from the output script to the input script. The output script is now simple and contains only three lines. The complexity is transferred to the redeem script. The output script only needs to provide the hash value of the redeem script, while the redeem script must provide the n public keys and the values of n and m . This redeem script is provided by the receiver.

Returning to the e-commerce example, the receiver is the e-commerce store. Therefore, the store only needs to publish the hash value of its redeem script on its website. The user then includes this hash value in the output script when generating the transfer transaction. Users do not need to know the details of the signature rules, such as whether three out of five or four out of six signatures are required. Therefore, this method is more convenient for users.

input script:

✗

```
PUSHDATA(Sig_1)
PUSHDATA(Sig_2)
...
PUSHDATA(Sig_M)
PUSHDATA(serialized RedeemScript)
```

redeemScript:

```
M
PUSHDATA(pubkey_1)
PUSHDATA(pubkey_2)
...
PUSHDATA(pubkey_N)
N
CHECKMULTISIG
```

output script:

```
HASH160
PUSHDATA(RedeemScriptHash)
EQUAL
```

Fig 70

Figure 71 shows the specific script execution process. First, the input and output scripts are combined. FALSE is the useless element. The two signatures are then pushed onto the stack, followed by the serialized redeem script. At this point, the execution of the input script is complete.

Next, the output script is executed. First, the hash value of the redeem script is calculated. The hash value provided in the output script is then pushed onto the stack. Finally, the two hash values are compared to determine whether they are equal. This completes the first phase of verification.

```
FALSE
PUSHDATA(Sig_1)
PUSHDATA(Sig_2)
PUSHDATA(serialized RedeemScript)
HASH160
PUSHDATA(RedeemScriptHash)
EQUAL
```

Fig 71

Next, we begin the second phase of verification by expanding and executing the redeem script (Figure 72). First, m=2 is pushed onto the stack. The three public keys are then pushed onto the stack, followed by n=3. Finally, the multi-signature is checked for validity.

```
2
PUSHDATA (pubkey_1)
PUSHDATA (pubkey_2)
PUSHDATA (pubkey_3)
3
CHECKMULTISIG
```

Fig 72

Proof of Burn

Finally, we are going to discuss a special script type: **the return script**. This output script begins with RETURN and can be followed by any content. The function of RETURN is to return false unconditionally, so a script containing this operation will never pass verification. Once RETURN is executed, the script terminates, and the content that follows it is not executed.

Why would we use such an output script? In fact, this script provides a way to prove that bitcoins have been burned. Burning bitcoins is mainly used in two situations. For example, you may need to burn some bitcoins to obtain some altcoins.

Another application takes advantage of the permanent storage feature of the Bitcoin blockchain to preserve information, such as content related to a particular piece of intellectual property. After hashing the content, a user can place its hash value after the RETURN statement. This hash value does not occupy much space and does not reveal the specific content of the intellectual property.

In this way, if someone claims in the future to be the author of the intellectual property and a dispute occurs, you can use this record to prove that you already knew the information at a particular point in time. This is similar to the coinbase field in a coinbase transaction, whose content can be chosen freely.

Why don't we simply use the coinbase field to implement this feature so that we do not need to burn bitcoins? This is because the coinbase field can be used only by nodes that have obtained the right to append a block. In contrast, any user can burn a small amount of bitcoin in exchange for the opportunity to write content to the blockchain and store it permanently.

Figure 73 shows a coinbase transaction with two outputs. The first output is a normal Pay-to-Public-Key-Hash output, and its amount is equal to the block reward plus the transaction fees. The amount of the second output is 0, and its output script uses the RETURN format. The purpose of this second output is to write information to the blockchain.

View information about a bitcoin transaction

090a9343da1158dce3634076f3df0f868ea777c97b3b2cf6f3d85ca7ea1f7b2

No Inputs (Newly Generated Coins)

18cBEMRxxHq... (ViaBTC Bitcoin Mining Pool) - (Spent)
Unable to decode output address - (Unspent)

12.65566873 BTC
0 BTC

12.65566873 BTC

Summary	
Size	257 (bytes)
Weight	920
Received Time	2018-07-05 12:34:39
Reward From Block	530572
Scripts	Hide scripts & Coinbase
Visualize	View Tree Chart

CoinBase

038c1808162f5669614254432f4d696e65642062792077676c312f2c1ab6d626a96a921da2cbb032a60989e13a368352565e31744b7137eaf6e191dc153040000000000000010ba10e
(decoded) □□□□/ViaBTC/Mined by wgl1/♦♦mm8♦♦♦♦.2♦♦♦♦♦♦♦♦.RV¹⁸1tKq7♦♦♦♦♦♦♦♦S♦♦♦♦♦♦♦♦♦♦♦♦♦♦♦♦♦x8

Output Scripts

DUP HASH160 PUSHDATA(20)[536ffa992491508dca0354e52f32a3a7a679a53a] EQUALVERIFY CHECKSIG

```
RETURN PUSHDATA(36)[aa21a9ed29817b27b64c8e79e0f0a7214fa67135872e2b8a5b5b1a6923f83a37c23bfc96]
(decoded) 0!00000f0L0y000!00q50.*0[[0#0:70:00
```



Fig 73

Figure 74 shows a regular transaction. As you can see, the output script begins with RETURN. The input amount of this transaction is 0.05 bitcoins, while the output amount is 0.

This transaction does not actually destroy any bitcoins. Instead, it uses the input bitcoins as a transaction fee and gives them to the miner who mines the block. When a miner sees this type of transaction, they know that its output can never be spent. Therefore, there is no need to store it in the UTXO set.

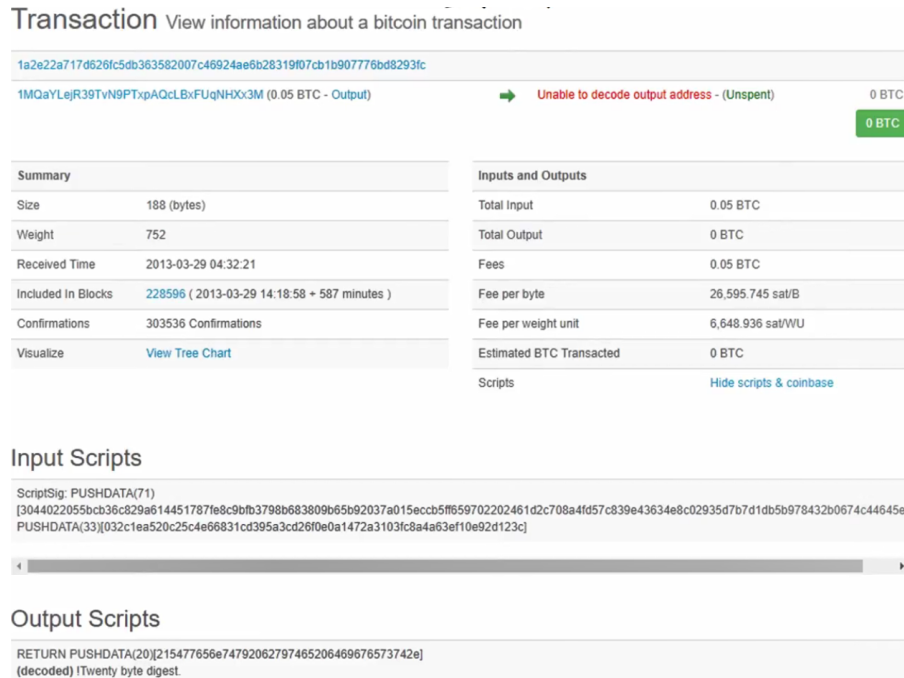


Fig 74

Bitcoin's scripting language is very basic, while Ethereum's smart contract language is much more complicated. Bitcoin's scripting language does not support loops, so its functionality is limited. However, this limitation is intentional. Because loops are not supported, infinite loops cannot occur, and there are no concerns about the halting problem.

Ethereum's smart contract language is Turing-complete, so a gas fee mechanism is needed to prevent a program from becoming trapped in an infinite loop.

Note that although Bitcoin's scripting language has limited functionality, it is very powerful for cryptographic applications. For example, CHECKMULTISIG, which verifies multiple signatures, can be implemented with a single instruction. This is much more convenient than using many general-purpose, high-level programming languages. Therefore, although Bitcoin's scripting language is simple, it is very well optimized.

Bitcoin Forking

A fork, as the name suggests, is a split in a blockchain. A fork may occur when two different nodes mine a block at the same time and both publish their blocks. This creates a temporary fork called a **state fork**, which is caused by a divergence in the state of the blockchain.

The forking attack mentioned earlier is also a state fork, but in this case, the divergence is intentionally created by malicious miners. Therefore, it is sometimes called a **deliberate fork**.

Another situation in which a fork may occur is when the Bitcoin protocol changes. Such a change requires an upgrade to the Bitcoin software. Because of Bitcoin's decentralized nature, we cannot guarantee that all nodes will adopt the upgrade. For example, some nodes may not have had enough time to upgrade, while others may not accept the new protocol and may be unwilling to upgrade.

A fork created in this situation is called a **protocol fork**. Depending on the nature of the protocol modification, a protocol fork can be further classified as either a **hard fork** or a **soft fork**.

Hard Fork

If the new protocol adds new features and functions, Bitcoin nodes that have not been upgraded will consider these features and functions invalid. For example, Bitcoin's block size is currently 1 MB, but some people believe that this is too small, limiting Bitcoin's throughput and increasing transaction delays.

One megabyte is 1,000,000 bytes, and each transaction is approximately 250 bytes. Therefore, each block can contain no more than approximately 4,000 transactions, while the average block time is 10 minutes. As a result, Bitcoin can process approximately seven transactions per second. This speed is indeed very slow, and credit card transactions can be processed much faster. Therefore, some people believe that such a small block size limits Bitcoin's transaction throughput and increases latency.

Now, suppose a Bitcoin software update increases the block size from 1 MB to 4 MB. Most nodes adopt this update, and we call them "new nodes." More precisely, this means that the nodes controlling most of the system's computing power have updated their Bitcoin software. The nodes controlling a minority of the computing power that have not been updated are called "old nodes" (Figure 75).

When the new nodes mine blocks, they produce large blocks of up to 4 MB. The old nodes do not recognize these large blocks, so they continue mining small blocks of up to 1 MB.

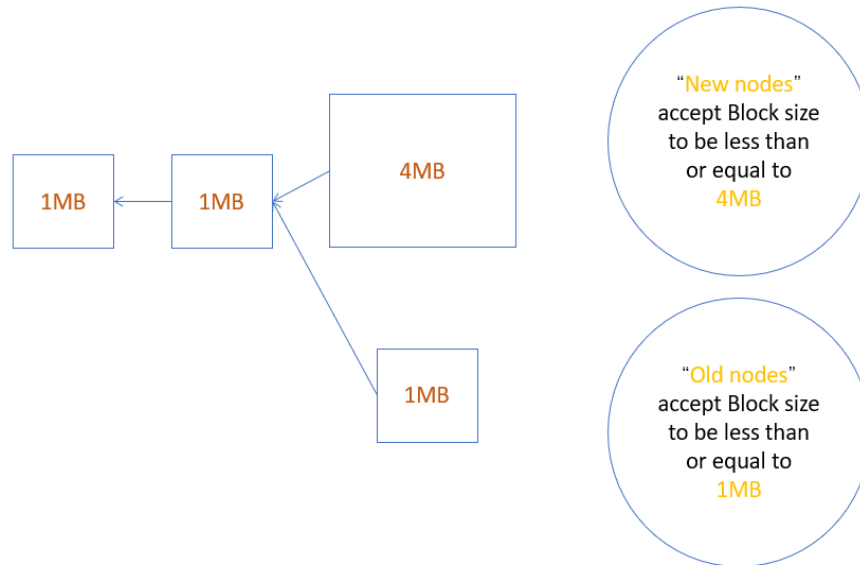


Fig 75

Note that the "new nodes" will accept the small blocks mined by the "old nodes" because the new protocol only stipulates that the block size cannot exceed 4 MB (Figure 76).

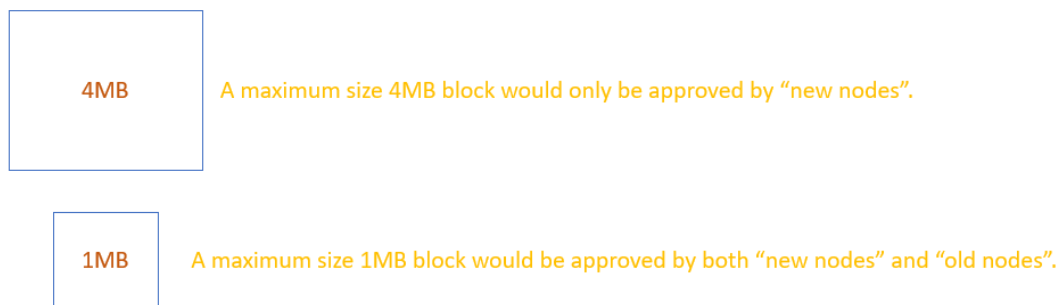


Fig 76

The current situation is that the new nodes, which control most of the computing power, will mine on top of the large blocks, while the old nodes, which control a minority of the computing power, will mine on top of the small blocks (Figure 77).

Although the new nodes consider both the upper and lower chains valid, the upper chain becomes the longest valid chain because of the new nodes' greater computing power. Therefore, the upper chain grows faster than the lower chain.

Note that the new nodes may also publish blocks smaller than 1 MB. For example, in Figure 77, the new nodes publish a 0.8 MB block, which can be accepted by both the old and new nodes.

However, the old nodes will not recognize the chain containing this block because the upper chain also contains blocks that they consider invalid—that is, blocks larger than 1 MB.

Therefore, this type of fork is permanent. As long as the old nodes do not update their software, the fork will not disappear. For this reason, it is called a **hard fork**.

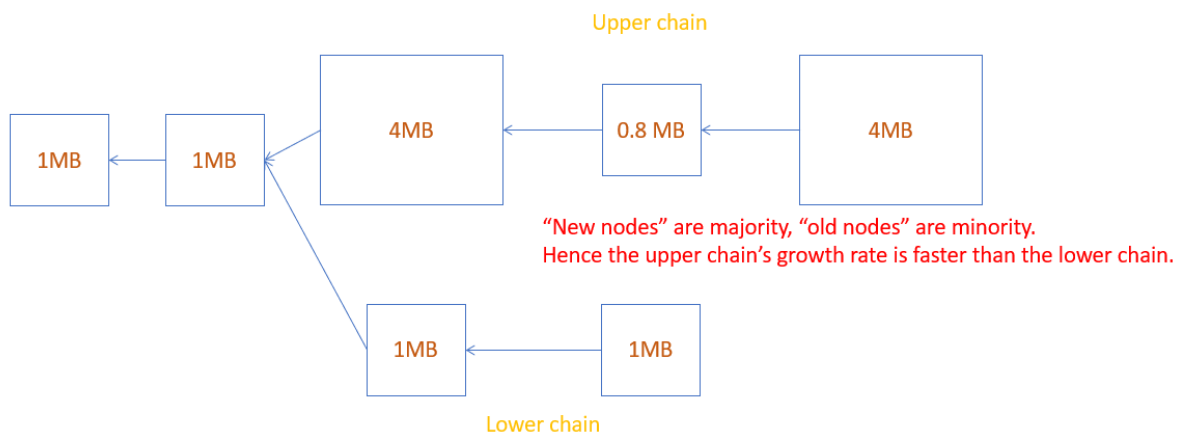


Fig 77

A larger block size is not necessarily better because Bitcoin's underlying network uses the P2P flooding model, which consumes a considerable amount of bandwidth. Bandwidth is the bottleneck.

After a hard fork occurs, the result is two chains running in parallel, each with its own cryptocurrency. It may also lead to a division within the community: some people may recognize the upper chain, while others may recognize the lower chain.

What happens after the fork to the coins that existed before the hard fork? These coins will be recognized by both the upper and lower chains. One original coin will therefore split into two coins. For example, the current ETH is the product of a hard fork, while the original Ethereum is ETC. Because of a hacker attack, the community decided to create ETH through a hard fork.

Let's use ETH and ETC as an example. After the hard fork, the two chains should maintain their own separate ledgers. However, because they were part of the same chain before the fork, users have the same private and public keys on both chains.

If no preventive measures are taken, a replay attack may occur. For example, suppose a transaction from B to C occurs on the ETH chain. Someone may replay the same transaction on the ETC chain, causing B to transfer not only 10 ETH to C but also 10 ETC (Figure 78).

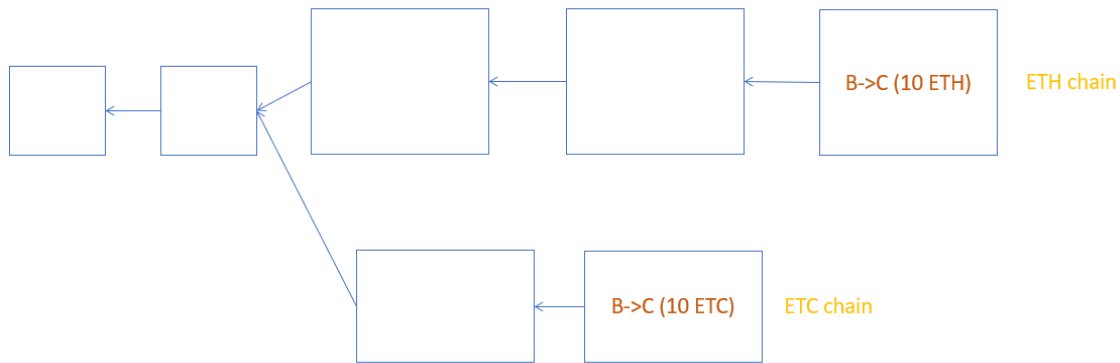


Fig 78

To prevent this type of replay attack, the two chains must use different chain IDs to distinguish themselves from each other after the hard fork.

Soft Fork

Next, let's discuss a soft fork. A soft fork occurs when new restrictions are added to the Bitcoin protocol, causing blocks and transactions that were previously valid to become invalid under the new rules.

Now, suppose a software update changes the block size limit from 1 MB to 0.5 MB. Recall that upgrading decentralized software is not as simple as changing a few parameters. The upgrade method may result in either a hard fork or a soft fork. This example is for illustration purposes only; no one wants to reduce the block size from 1 MB to 0.5 MB because 1 MB is already quite small.

We still assume that the new nodes represent the majority and accept the software update that reduces the block size limit to 0.5 MB (Figure 79).

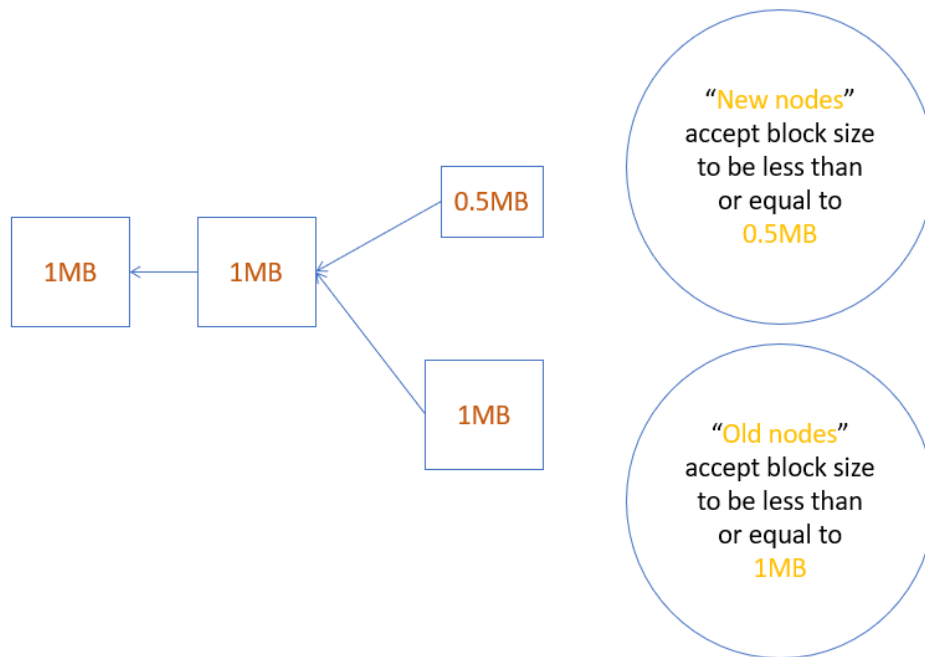


Fig 79

After the upgrade, the new nodes begin mining smaller blocks of up to 0.5 MB, which are also accepted by the old nodes. The old nodes continue mining blocks of up to 1 MB, which are not accepted by the new nodes (Figure 80).

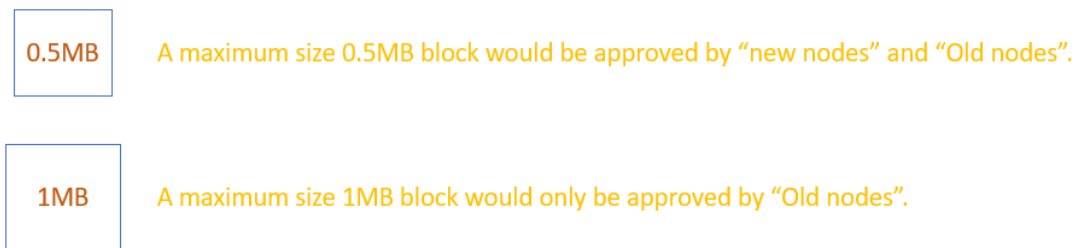


Fig 80

Although the old nodes accept both the upper and lower chains, the Bitcoin protocol requires them to extend the longest valid chain. Because the new nodes have greater computing power, the upper chain grows faster. This causes the old nodes to abandon their current fork, switch to the longest valid chain, and continue mining on top of it (Figure 81).

The old nodes then mine another block of up to 1 MB. The new nodes do not accept this new 1 MB block, so the blockchain forks again, and the process repeats. Therefore, if the old nodes do

not update their software, they will continue mining in vain while repeatedly creating temporary forks.

This situation is temporary and does not create two chains that run in parallel, as a hard fork does. Therefore, this type of fork is called a **soft fork**.

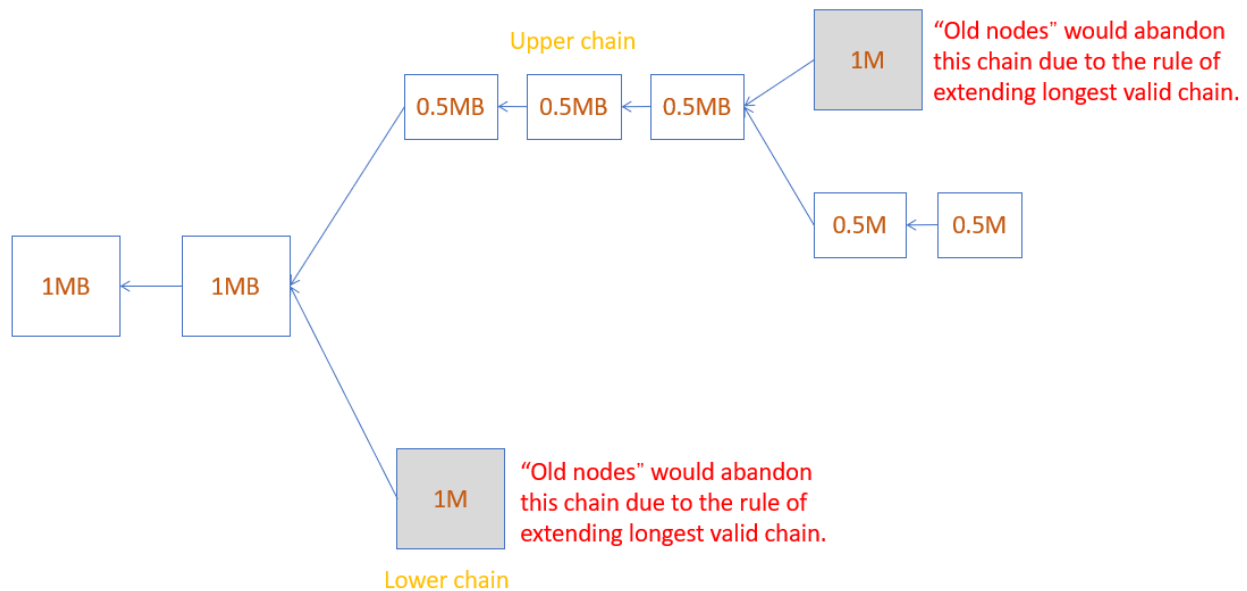


Fig 81

A possible soft fork could involve assigning new meanings and rules to previously unspecified fields in the current protocol, such as the coinbase field. Currently, there are no rules governing this field, and no one checks its contents.

We previously discussed how this field can be used as an extra nonce field for mining. Without this extra nonce, the search space provided by the original nonce field is only 4 bytes, or 2^{32} possible values. This search space is not large enough, so we use the first 8 bytes of the coinbase field as an extra nonce to increase the total search space to 2^{96} .

However, the coinbase field is longer than 8 bytes and has additional available space. Therefore, some people have proposed using this extra space to store the root hash value of the UTXO set. Currently, each full node maintains the UTXO set in memory so that it can quickly determine whether a transaction involves double-spending. The contents of this set are not actually written to the blockchain.

Why did people make this proposal? As mentioned earlier, a light node does not need to store the entire blockchain; it only needs to store the block headers. A light node verifies whether a transaction is included in a particular block by requesting a Merkle proof from a full node.

However, if we want to determine how many bitcoins a particular account has, the current Bitcoin system cannot actually provide proof of the balance. A full node can calculate how many bitcoins an account has by examining the account-related transaction information in the UTXO set. However, many blockchain wallets operate only as light nodes. Therefore, if a light node wants to know an account's balance, it can only ask a full node to perform the calculation.

However, how can the light node verify that the answer provided by the full node is correct? The answer is that it cannot verify it. If the light node does not maintain the UTXO set, it cannot independently verify the answer.

Therefore, some people have proposed forming a Merkle tree from the contents of the UTXO set and then writing the root hash of this tree into the extra space in the coinbase field. This is possible because the contents of the coinbase field affect the root hash value in the block header. In this way, a Merkle proof could be used to prove the balance of an account.

If this proposal were adopted, it would cause a soft fork. The blocks published by the new nodes would be accepted by the old nodes because the old nodes do not care about the contents of the coinbase field. However, the new nodes would not accept blocks published by the old nodes because the old nodes would not place the root hash of the UTXO Merkle tree in the coinbase field.

A well-known example of a soft fork in Bitcoin's history is P2SH. This feature was not initially available but was later added through a soft fork.

In summary, after a hard fork, the new nodes accept blocks mined by the old nodes, while the old nodes do not accept blocks mined by the new nodes. This results in permanent parallel chains. After a soft fork, the new nodes do not accept blocks mined by the old nodes, while the old nodes accept blocks mined by the new nodes. This results in temporary parallel chains.

Bitcoin Anonymity

Although you do not need to provide your real name when generating a public-private key pair for a Bitcoin account, it is difficult to achieve complete anonymity with Bitcoin. Cash transactions are truly anonymous.

If a bank did not require customers to open accounts under their real names, anyone with an account password could withdraw money. In that case, the bank's services would actually provide greater anonymity because its ledger would be private. In contrast, the Bitcoin ledger is completely public.

Anonymity Sabotage

Some people suggest generating a new public-private key pair every time you receive bitcoins so that others cannot determine which addresses belong to you. But can this method truly protect your privacy? Not necessarily, because in some cases, the different accounts you have created can be linked.

As mentioned earlier, each transaction can have multiple inputs and outputs. Suppose a transaction like the one shown in Figure 82 is published on the blockchain. It is very likely that `addr1` and `addr2` belong to the same person because that person must control the private keys of both accounts.

Why are there multiple inputs? The item you want to purchase with bitcoin may cost more than the balance in a single account, so you may need to use multiple accounts to pay for it.

In addition, one of the output addresses shown below is likely to be a change address. Such transactions are generally generated by Bitcoin wallet software. To protect privacy, many wallet applications generate a new change address for every transaction.

Is it still possible to associate the input and output addresses with one another? There may be n input addresses. Can we determine whether all n addresses belong to the same person? Can we also identify the change address?

Inputs:	<code>addr1</code>	<code>addr2</code>
Outputs:	<code>addr3</code>	<code>addr4</code>

Fig 82

The answer to these questions is yes. For example, in Figure 83, it is clear that addr4 is the change address. Therefore, if we want stronger privacy protection, we can deliberately generate unnecessary outputs.

There are only a few commonly used Bitcoin wallets. Therefore, if you understand how these wallets operate, you can analyze transactions throughout the Bitcoin blockchain because these wallets rarely provide a feature for deliberately generating unnecessary addresses.

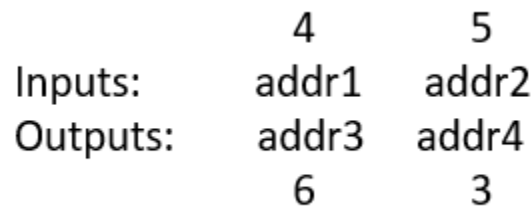


Fig 83

The case discussed above involves a person attempting to increase their anonymity by generating multiple addresses. However, through our analysis and discussion, we found that it may still be possible to link all the person's addresses.

Next, let's discuss the connection between your Bitcoin address and your real-world identity. When can others determine that a particular Bitcoin address corresponds to a particular person in real life? This can happen whenever there is a connection between Bitcoin and the physical world.

If you operate only within the blockchain system, it is difficult for others to determine your real-world identity. However, once the physical world becomes involved, such as when fiat currency is transferred into or out of the blockchain system, your identity may be revealed. For example, when we use dollars to buy bitcoins, we generally purchase them through an exchange, which first verifies our real-world identities.

Even if you use an over-the-counter transaction—a private transaction between individuals—or gather with other people in a random park to trade, your identity may still be revealed. To prevent Bitcoin from being used for money laundering, many countries closely monitor funds entering and leaving the Bitcoin system. For example, if you want to convert millions of dollars into bitcoins or a large number of bitcoins into dollars, the transaction is unlikely to go unnoticed by law enforcement authorities.

Some merchants now accept Bitcoin payments directly, but does this guarantee anonymity and privacy? A study was once conducted to examine the credit card usage habits of different groups of people. An organization collected their transaction information, hashed the card

numbers, and removed the cardholders' identities. It then published the transaction information.

It took some time, but people eventually analyzed the hashed transaction records and discovered the cardholders' identities. How did they do it? For example, suppose A knows B and happens to see B purchase something from a small store at 2:00 p.m. Of course, A does not know B's card number. However, when the transaction records are published after being hashed, A can filter many of the transactions based on the knowledge that B made a purchase at 2:00 p.m.

The next time A sees B purchase something from a particular store at a particular time, A can filter the records again and narrow down B's identity further. After repeating this process several times, A may be able to connect the hashed card number to B. This example shows that even when credit card numbers are hashed, people may still be able to determine which number belongs to which person.

Bitcoin transaction information is also public. If you use the same Bitcoin account to pay at Starbucks several times, people may be able to determine that the account belongs to you.

In summary, different accounts belonging to the same person can be linked through analysis within the Bitcoin system. When Bitcoin interacts with the real world, the person's identity may also be discovered. Therefore, Bitcoin does not provide a very high level of anonymity.

The person who has achieved the highest level of Bitcoin anonymity is Satoshi Nakamoto. Satoshi participated in Bitcoin for a long time, but the bitcoins they obtained have never been spent. If Satoshi were to spend those bitcoins, the transaction would have to interact with the real world, potentially revealing Satoshi Nakamoto's true identity.

There was once a website called Silk Road that was used to buy and sell illegal goods. It was essentially the eBay of the dark web. To evade law enforcement, the website used Bitcoin as its payment method and the Onion network as its underlying network. Anonymous mailing services were also used for shipping.

After operating for several years, the website was shut down by the government, and its owner was arrested. This person could not spend his bitcoins because doing so would reveal his identity.

Anonymity Reinforcement

What methods can ordinary Bitcoin users use to improve their anonymity? As mentioned earlier, Bitcoin operates at the application layer, while its underlying network is a P2P network. Therefore, we can improve anonymity at both of these layers.

For example, suppose we visit an Internet café and use a screen name. Is there any way for others to discover our identity? Of course, we usually have to register with an ID when entering

an Internet café, and our location may also be identified through our IP address. An IP address is often closely connected to a person's real-world identity.

Therefore, we must first find a way to achieve anonymity at the network layer. The good news is that effective solutions for network-layer anonymity already exist. We can use multi-path forwarding methods, such as Tor onion routing. Each node that forwards a message knows only the previous node and does not know who originally sent the message.

Next, let's discuss how to improve anonymity at the application layer. As mentioned earlier, because Bitcoin uses an open ledger, every transaction can be traced. Even if someone creates many Bitcoin accounts, we may still be able to link those accounts to the same person.

However, we can use a coin-mixing method to make this more difficult. Coin mixing involves mixing the coins of different people. Some websites provide coin-mixing services. Everyone sends coins to the website and later receives coins in return. Generally, the coins you receive are not the same coins you originally sent. However, current coin-mixing services may not be trustworthy, and they may run away with your coins.

Another similar approach does not require us to perform coin mixing intentionally because some applications naturally have a coin-mixing effect. Online wallets are one example. Everyone deposits their coins, and when you later retrieve yours, they may not be the same coins you originally deposited. Online wallets can therefore have a coin-mixing effect, although there is no guarantee that the coins will always be mixed.

You can also use cryptocurrency exchanges because most exchanges naturally have a coin-mixing effect. For example, suppose you have 10 bitcoins. You send them to an exchange and convert them into stablecoins. You then use the stablecoins to purchase litecoins or other cryptocurrencies before eventually buying bitcoins again. Finally, you withdraw these bitcoins to your local wallet. Generally, the bitcoins you withdraw are not the same bitcoins you originally deposited.

Of course, this method assumes that the exchange will not disclose your transaction records to the government. In fact, maintaining privacy on the Bitcoin blockchain is quite difficult because Bitcoin uses an open ledger, and information recorded on the blockchain is very difficult to change. Its immutability and public nature create serious challenges for privacy protection.

Zero-knowledge proof & Blind Signature

Let's discuss Zero-knowledge proof (Fig 84)

Zero-knowledge proof

From Wikipedia, the free encyclopedia

"ZKP" redirects here. For the airport in Russia, see Zyranka Airport. For other uses, see Zero knowledge.

In cryptography, a **zero-knowledge proof** or **zero-knowledge protocol** is a method by which one party (the prover) can prove to another party (the verifier) that a given statement is true while the prover avoids conveying any additional information apart from the fact that the statement is indeed true. The essence of zero-knowledge proofs is that it is trivial to prove that one possesses knowledge of certain information by simply revealing it; the challenge is to prove such possession without revealing the information itself or any additional information.^[1]

Fig 84

For example, I, the prover, want to prove to you, the verifier, that a particular Bitcoin account belongs to me. However, I cannot reveal my private key. How can I prove that the account is mine?

I can generate a signature and provide it to you. If you know the account's public key, you can use it to verify my signature. However, whether this example qualifies as a zero-knowledge proof is debatable because I have disclosed a signature generated using the private key.

Zero-knowledge proofs are based on homomorphic hiding (Figure 85).

Homomorphic hiding

- If $x \neq y$, then $E(x) \neq E(y)$.
- Given the value of $E(x)$, it is almost impossible to get x .
- Given the values of $E(x)$ and $E(y)$, we can easily calculate the $E()$ value of $x + y$, $x * y$, etc.

Fig 85

The first property indicates that collisions cannot occur. The hash functions discussed at the beginning may produce collisions. In homomorphic hiding, if the input x is not equal to y , then $E(x)$ and $E(y)$ must also be different. This also means that if $E(x)$ and $E(y)$ are equal, then x and y must be equal.

The second property indicates that the encryption function is irreversible. It is difficult to infer the value of x from the value of $E(x)$. This is similar to the hiding property of a hash function.

The third property is homomorphic operation. This means that performing algebraic operations on encrypted values is equivalent to performing the algebraic operations first and then encrypting the result.

Figure 86 shows an example of a zero-knowledge proof.

Example

- Alice wants to prove to Bob that she knows a set of x and y so that $x + y = 7$. Alice can not tell Bob the value of x and y .

Fig 86

Figure 87 shows a simple solution.

Simple solution

- **Alice provide $E(x)$ and $E(y)$ to Bob.**
- **Bob can calculate $E(x + y)$ by $E(x) + E(y)$.**
- **Bob then calculates $E(7)$, if $E(7) = E(x + y)$ verification is true.**

Fig 87

However, this method is flawed because Bob can determine the values of x and y through brute force. Therefore, Alice should introduce some randomness before giving $E(x)$ and $E(y)$ to Bob. However, Alice must ensure that the value of $(x+y)$ remains unchanged.

Recall that we discussed a centralized digital currency design in which the central bank maintains a database to keep track of each signed and serialized digital currency file. This model is centralized because the central bank must participate in verifying every transaction. Therefore, the central bank knows all your transaction information.

Is there a way for the central bank to maintain this centralized ledger and check for double-spending without knowing the specific parties involved in each transaction?

Yes, this is possible. However, the central bank cannot generate the serial number of each digital currency file. Instead, users must generate the serial numbers of these digital currency files locally without revealing them to the central bank.

For example, A wants to transfer money from their deposit at the central bank to B, but A does not want the bank to know that B's money came from A. A blind-signature method can be used:

- In the first step, A generates the serial number locally. The bank then signs it without knowing the specific serial number and deducts the amount from A's balance.
- In the second step, A gives B the actual serial number and the signature created by the bank in the first step.
- In the third step, B gives the bank the actual serial number and the signature received from A. The bank then performs the verification—that is, it checks for double-spending. After the verification succeeds, the bank increases B's balance.

Reference

Blockchain and Application, Beijing University, Professor Zhen Xiao, <http://zhenxiao.com>

Computer Network, University of Southern California, Professor Shahin Nazarian, [Shahin Nazarian, PhD \(usc.edu\)](#)

Blockchain from Beginning to Master, BlockChainSaw, [\(1711\) BlockChainSaw - YouTube](#)

[What is a Digital Signature? \(youdzone.com\)](#)

xuanf@usc.edu